



Universidad Internacional de La Rioja
Escuela Superior de Ingeniería y Tecnología

Grado en Ingeniería Informática

Sistema experto para la toma de decisiones estratégicas en League of Legends

Trabajo fin de estudio presentado por:	Oscar Sánchez García
Director/a:	Claudia Blanca Gonzalez Calleros
Fecha:	17/06/2026
Repositorio del código fuente:	https://github.com/OSZ8/sistema-experto-tfg-unir

Resumen

El presente Trabajo Fin de Grado describe el diseño, desarrollo y validación de un sistema experto híbrido orientado a la toma de decisiones estratégicas durante la fase de selección de personajes (draft) en el videojuego competitivo League of Legends. Las herramientas comerciales vigentes en el sector se limitan a ofrecer análisis estadísticos masivos de carácter descriptivo, lo que satura cognitivamente al usuario y carece de un contexto causal que justifique las recomendaciones. Para resolver esta problemática, se ha construido una solución de software que combina el rigor del modelado lógico-simbólico con la frescura de las métricas de rendimiento deportivo en tiempo real.

El núcleo del sistema consiste en un motor de inferencia desarrollado a medida en Python, el cual implementa un algoritmo determinista de encadenamiento hacia adelante (forward chaining) respaldado por una ontología estructural de etiquetas semánticas y reglas de producción prioritarias. Para garantizar la viabilidad operativa bajo restricciones de tiempo real, el backend se apoya en una arquitectura de persistencia local optimizada en SQLite que actúa como caché de datos masivos frente a las limitaciones de tráfico (rate limiting) impuestas por la API oficial de Riot Games. La interfaz de usuario se ha implementado mediante el framework Flask, delegando la visualización en una capa frontal asíncrona que manipula dinámicamente el DOM a través de JavaScript, reduciendo la latencia y la sobrecarga de red.

Un aporte diferencial del proyecto radica en la integración nativa de la Inteligencia Artificial Explicable (XAI). Mediante el rastreo dinámico de las fuentes de evidencia en la memoria de trabajo, el sistema es capaz de romper el efecto de "caja negra" tradicional, formateando trazas de justificación en lenguaje natural que explican de manera transparente el porqué de cada sugerencia. Las pruebas empíricas de estrés y validación arrojaron un tiempo medio de respuesta de 0.42 segundos y una convergencia del 90% con el criterio de analistas humanos expertos, certificando la madurez técnica, la eficiencia y el valor pedagógico de la solución informática alcanzada.

Palabras clave: Sistema Experto, Motor de Inferencia, Encadenamiento hacia Adelante, Inteligencia Artificial Explicable (XAI), Flask, Arquitectura Asíncrona, League of Legends.

Abstract

This Bachelor's Thesis describes the design, development, and validation of a hybrid expert system tailored for strategic decision-making during the character selection phase (draft) in the competitive video game League of Legends. Current commercial tools in the sector are restricted to providing massive descriptive statistical analysis, which overloads the user cognitively and lacks a causal context to justify the recommendations. To solve this problem, a software solution has been built that combines the rigor of symbolic-logical modeling with the freshness of real-time sports performance metrics.

The core of the system consists of a custom inference engine developed in Python, which implements a deterministic forward chaining algorithm backed by a structural ontology of semantic tags and prioritized production rules. To guarantee operational viability under real-time constraints, the backend relies on an optimized local persistence architecture in SQLite that acts as a massive data cache against the traffic restrictions (rate limiting) imposed by the official Riot Games API. The user interface has been implemented using the Flask framework, delegating the visualization to an asynchronous frontend layer that dynamically manipulates the DOM through JavaScript, reducing latency and network overhead.

A differential contribution of this project lies in the native integration of Explainable Artificial Intelligence (XAI). By dynamically tracking the sources of evidence in the working memory, the expert system can break the traditional "black box" effect, formatting justification traces in natural language that transparently explain the reasoning behind each suggestion. Empirical stress and validation tests yielded an average response time of 0.42 seconds and a 90% convergence rate with the criteria of expert human analysts, certifying the technical maturity, efficiency, and pedagogical value of the achieved computing solution.

Keywords: Expert System, Inference Engine, Forward Chaining, Explainable Artificial Intelligence (XAI), Flask, Asynchronous Architecture, League of Legends.

Índice de contenidos

1.	Introducción	1
1.1.	Motivación	3
1.2.	Conceptos básicos y dominio del problema	5
1.2.1.	Ontología de etiquetas y roles.....	6
1.2.2.	El “hecho” (fact) y la explicabilidad	7
1.2.3.	La fase de selección (draft) y las reglas de producción	7
1.3.	Estructura del trabajo.....	11
2.	Contexto y Estado del Arte	12
2.1.	Análisis del contexto estratégico y complejidad del dominio	13
2.1.1.	La magnitud del problema: El espacio de decisiones	13
2.2.	Estado del arte: soluciones actuales y su evolución	14
2.2.1.	Análisis de herramientas comerciales: El predominio de la estadística descriptiva	14
2.2.2.	Evolución de la IA en los eSports: De la predicción a la explicación	15
2.2.3.	El sistema experto como alternativa pedagógica.....	15
2.2.4.	Evaluación crítica y matriz comparativa de enfoques tecnológicos.....	16
3.	Objetivos y metodología de trabajo.....	19
3.1.	Objetivo general	19
3.2.	Objetivos específicos	20
3.3.	Metodología de trabajo.....	21
3.3.1.	Enfoque Ágil: Desarrollo Iterativo e Incremental.....	21
3.3.2.	Cronograma de Trabajo y Diagrama de Gantt.....	23
3.3.3.	Herramientas de Gestión del Proyecto	24
4.	Análisis y Diseño de la Solución.....	25

4.1.	Análisis de requisitos	25
4.1.1.	Identificación de actores	25
4.1.2.	Requisitos funcionales (RF).....	26
4.1.3.	Requisitos no funcionales (RNF)	27
4.1.4.	Análisis de casos de uso.....	28
4.1.5.	Análisis de restricciones del sistema	30
4.1.6.	Matriz de trazabilidad de requisitos.....	31
4.2.	Arquitectura del sistema	32
4.2.1.	Vista General de la Arquitectura	32
4.2.2.	Diseño de Componentes del Backend	33
4.2.3.	Integración con Servicios Externos y Almacenamiento Local	34
4.2.4.	Ciclo de Vida de una Petición de Recomendación	35
4.3.	Diseño del Modelo de Datos	35
4.3.1.	Estructura de la Ontología de Conocimiento (JSON).....	35
4.3.2.	Modelado de Clases del Sistema Experto.....	36
4.3.3.	Persistencia de Datos Estadísticos (SQLite)	38
4.3.4.	Interoperabilidad y Mapeo de Atributos	38
4.4.	Lógica del Motor de Inferencia y Algoritmo de Decisión	39
4.4.1.	Proceso de Inferencia y Resolución de Conflictos.....	39
4.4.2.	Algoritmo de Scoring Híbrido: Lógica vs. Estadística.....	40
4.4.3.	Implementación de la Explicabilidad (XAI)	40
4.5.	Plan de Pruebas y Validación del Sistema	41
4.5.1.	Pruebas de Rendimiento y Eficiencia Computacional (Tiempos de Respuesta)	41
4.5.2.	Validación en Entornos Reales: Escenario de Prueba Crítico.....	42

4.5.3.	Análisis de Convergencia y Explicabilidad (XAI).....	43
5.	Implementación del Sistema	44
5.1.	Entorno de Desarrollo y Herramientas Técnicas.....	44
5.1.1.	Lenguaje de Programación Base: Python	44
5.1.2.	Framework Web y Motor de Plantillas.....	45
5.1.3.	Entorno de Trabajo y Control de Versiones	46
5.2.	Integración con la API de Riot Games y Almacenamiento Local.....	46
5.2.1.	Consumo de Servicios Web: Data Dragon y Endpoints Dinámicos	47
5.2.2.	Estrategia de Caché y Persistencia en SQLite contra el Rate Limiting	47
5.2.3.	Codificación del Módulo de Sincronización.....	48
5.3.	Desarrollo del Backend (Motor de Reglas).....	49
5.3.1.	Estructura Lógica: Clases Fact y Rule.....	49
5.3.2.	Implementación del Algoritmo de Inferencia (Forward Chaining).....	50
5.4.	Desarrollo de la Interfaz de Usuario y Capa de Presentación (Frontend)	52
5.4.1.	Arquitectura de Plantillas con Jinja2	53
5.4.2.	Estilo Visual y Experiencia de Usuario (CSS)	53
5.5.	Manual de Despliegue y Configuración del Entorno Local.....	54
5.5.1.	Requisitos Previos e Instalación de Dependencias.....	54
5.5.2.	Configuración de Variables de Entorno y Seguridad	55
5.5.3.	Inicialización del Servidor y Verificación Ejecutiva	56
6.	Conclusiones y Trabajo Futuro	57
6.1.	Conclusiones del Trabajo.....	57
6.2.	Líneas de Trabajo Futuro	59
	Referencias bibliográficas.....	61

Índice de figuras

Figura 1. Mapa de la Grieta del Invocador con las líneas (Top, Mid, Bot y Jungla).....	5
Figura 2. Interfaz del recomendador pidiendo al usuario insertar los datos de los campeones de la partida actual	8
Figura 3. Interfaz del recomendador tras un usuario haber insertado los personajes de la partida.....	9
Figura 4. El programa da como resultado su conclusión tras realizar los calculos y recomienda las mejores opciones	10
Figura 5. Diagrama de Gantt reflejando los Sprints de desarrollo del proyecto	24
Figura 6. Definición técnica de la clase Fact para la gestión de evidencias en la memoria de trabajo.....	37
Figura 7. Implementación de la clase Rule para la definición de la lógica de producción del sistema.....	37
Figura 8. Dependencias y librerías clave configuradas en el entorno virtual del proyecto	46
Figura 9. Módulo de conexión con la API de Riot Games para la extracción dinámica de métricas de partidas	49
Figura 10. Implementación en Python de las estructuras base del conocimiento	50
Figura 11. Algoritmo de encadenamiento hacia adelante y motor de resolución con reemplazo dinámico de evidencias para la capa XAI.....	52
Figura 12. Estructura de contenedores en HTML para la inyección asíncrona de resultados ..	53
Figura 13. Comandos secuenciales para la clonación, inicialización e instalación del proyecto	55
Figura 14. Consola de comandos reflejando el arranque exitoso del servidor Flask en el entorno local.....	56

Índice de tablas

Tabla 1. Ontología de Etiquetas del Sistema Experto.....	6
Tabla 2. Matriz comparativa de enfoques tecnológicos para la asistencia en el draft	16
Tabla 3. Especificación del Caso de Uso CU-01: Consultar Recomendación de Draft.....	28
Tabla 4. Especificación del Caso de Uso CU-02: Sincronización de Datos Estadísticos	30
Tabla 5. Matriz de Trazabilidad entre Requisitos Funcionales y Objetivos del Proyecto	31
Tabla 6. Especificación de campos en la ontología de campeones	36
Tabla 7. Esquema de la base de datos de rendimiento estadístico	38
Tabla 8. Métricas cuantitativas del rendimiento del backend.	42

1. Introducción

La industria de los videojuegos ha sufrido cambios constantes en los últimos años. Antiguamente solo se percibía a los videojuegos como una fuente de entretenimiento, hoy en día, muchos de los títulos que solamente trataban de cumplir la función de entretener al usuario se han ido profesionalizando con el tiempo, llegando a convertirse en deportes electrónicos o eSports, comparables incluso a los deportes tradicionales. Dentro de este contexto, los juegos del género MOBA (Multiplayer Online Battle Arena), y en especial League of Legends (LoL), destacan como un escenario muy interesante para aplicar conceptos de ingeniería informática y, en particular, de inteligencia artificial y análisis de datos.

El propósito del trabajo no es únicamente la creación de otra herramienta más de consulta, pues en el mercado actual hay varias ofertas que cumplen con el propósito del presente trabajo, aunque presentan limitaciones individuales. Se pretende analizar cómo los sistemas basados en conocimiento pueden ayudar a poner orden en un entorno donde la información es incompleta, cambiante y bastante compleja. En definitiva, el proyecto busca organizar un entorno de variables constantes para buscar la mejor solución a problemas que se dan en la partida.

A través de la formación académica y la observación del dominio respecto al League of Legends, se ha constatado algo que se repite bastante: aunque muchos jugadores tienen un nivel mecánico muy alto, les falla la toma de decisiones estratégicas. Esto ocurre sobre todo en momentos clave como la selección de campeones (draft), la cual puede llegar a determinar gran parte del resultado de la partida sin siquiera jugar esta misma. De hecho, estudios sobre predicción en juegos competitivos sugieren que la fase de selección puede condicionar el éxito del encuentro de manera determinante antes de que los jugadores entren al mapa (Yang et al., 2016).

Para ello, se propone el desarrollo de un sistema experto que funcione como una especie de asesor estratégico. Se plantea que el sistema emule el razonamiento de un analista humano experto en la materia, pero aprovechando la capacidad de cálculo de un sistema informático, su velocidad y sin las limitaciones típicas como el cansancio o la presión.

Este proyecto no es relevante solo desde el punto de vista del juego. A nivel más teórico, League of Legends plantea un problema muy interesante relacionado con la teoría de juegos aplicada. Algunos estudios señalan que la dificultad no está únicamente en lo que ocurre dentro de la partida, sino también en todo el conocimiento que rodea al juego, lo que se conoce como “metagame” (Mora-Cantallops & Sicart, 2018). Esta postura se fundamenta en el hecho de que, con el tiempo, la escena competitiva dentro del mundo de los videojuegos se ha profesionalizado y se han estandarizado posiciones de coach (entrenador), analistas de draft, etc., mostrando que a nivel competitivo la toma de decisiones es relevante a los mismos niveles que saber jugar al juego con unas mecánicas profesionales.

Durante el proceso de desarrollo, han surgido diversos desafíos técnicos importantes. Por ejemplo, trabajar con datos en tiempo real usando las APIs oficiales de Riot Games (2024), o diseñar un sistema en Python capaz de no solo dar recomendaciones, sino también explicar por qué las da. Esto último es especialmente relevante, ya que muchos modelos actuales de inteligencia artificial funcionan como “cajas negras” y no ofrecen una justificación clara de sus decisiones, lo que dificulta que el usuario confíe plenamente en el sistema (Russell & Norvig, 2021).

De hecho, se ha optado por divergir ligeramente del enfoque basado únicamente en Machine Learning, que es el más utilizado hoy en día. Aunque estos modelos son muy buenos prediciendo resultados, suelen fallar en algo clave: la explicabilidad. En el presente trabajo se ha priorizado dicho aspecto, construyendo un sistema que no solo indique qué hacer, sino que también explique el motivo, basándose en factores como las sinergias entre campeones, los enfrentamientos directos o el equilibrio del equipo y sus necesidades.

Por último, esta introducción sirve como punto de partida para entender el resto del documento. En ella se detalla tanto la motivación detrás del proyecto como los conceptos básicos necesarios para seguirlo, incluso si no se tiene experiencia previa con este tipo de juegos competitivos. Asimismo, se presenta la estructura del trabajo, que dará paso a un análisis más técnico donde se detallará cómo se ha desarrollado el sistema. En esencia, este proyecto intenta dar forma y sentido al caos estratégico de una partida de League of Legends, utilizando herramientas propias de la lógica computacional.

1.1. Motivación

La elección de este tema para el Trabajo de Fin de Grado no es simplemente un interés personal por el videojuego y su contenido; reside también en que la experiencia competitiva previa en el dominio permite aportar un valor analítico adicional al desarrollo del proyecto, más allá de la perspectiva técnica. Como estudiante y seguidor de la escena competitiva de League of Legends desde hace años, resulta de gran interés observar la brecha existente entre la ejecución técnica y la estrategia pura. Se ha constatado cómo, de manera frecuente, partidas protagonizadas por jugadores con un alto nivel mecánico se pierden debido a una planificación inicial deficiente o a una composición de equipo inadecuada para las exigencias del encuentro, situaciones que se repiten de forma recurrente en el entorno competitivo.

La motivación principal del presente trabajo se fundamenta en la observación de que, ante la complejidad del juego, una gran parte de los usuarios suele tomar decisiones de manera intuitiva o basados en tendencias, careciendo a menudo de una comprensión profunda de la lógica subyacente. Tras investigar las herramientas de asistencia disponibles en la actualidad, se constata que la mayoría se limita a proporcionar porcentajes de victoria o listas de objetos recomendados a partir de estadísticas masivas. No obstante, tales soluciones carecen de una explicación sobre el razonamiento estratégico subyacente. Desde la perspectiva de la ingeniería, se plantea el desafío de diseñar un sistema innovador que no solo recomiende una elección, sino que sea capaz de razonar dicha decisión de manera análoga a un analista profesional.

En el plano tecnológico, se plantea el reto de divergir de las tendencias actuales. Hoy en día parece que cualquier solución de inteligencia artificial debe pasar obligatoriamente por el Machine Learning, pero estos modelos a menudo funcionan como cajas negras donde es imposible entender el porqué de un resultado. La finalidad de este trabajo es reivindicar la utilidad de los sistemas expertos en entornos estratégicos. Representa un desafío técnico relevante el desarrollo desde cero de un motor de inferencia en Python que fuera capaz de procesar reglas lógicas creadas por humanos. En última instancia, el objetivo es evitar que el usuario deba aceptar recomendaciones sin una fundamentación lógica explícita. Se considera de mayor interés el diseño de una arquitectura con lógica abierta, que el programa no se limite a dar una respuesta, sino que sea capaz de explicarle al jugador el porqué de cada decisión de una forma que tenga sentido y sea fácil de seguir. Dicha propuesta aporta un valor significativo para los usuarios noveles, quienes a menudo experimentan una carga cognitiva elevada ante la multiplicidad de variables del juego. A diferencia de las herramientas convencionales, que suelen proporcionar instrucciones directas sin una justificación estratégica clara, este sistema facilita la comprensión profunda del proceso de toma de decisiones.

Otro gran motor de este proyecto ha sido el reto de la implementación real. Se ha evitado que el trabajo se limite a un estudio teórico, estableciendo como meta el desarrollo de una aplicación funcional mediante el uso de Flask para la creación de una interfaz web intuitiva y profesional. La resolución de problemas técnicos reales, tales como la gestión de las APIs de Riot Games, la actualización constante de los datos del juego o el diseño de un sistema de degradación elegante para asegurar la estabilidad de la interfaz ante fallos de carga de recursos ha constituido una de las fases más significativas del proceso de desarrollo.

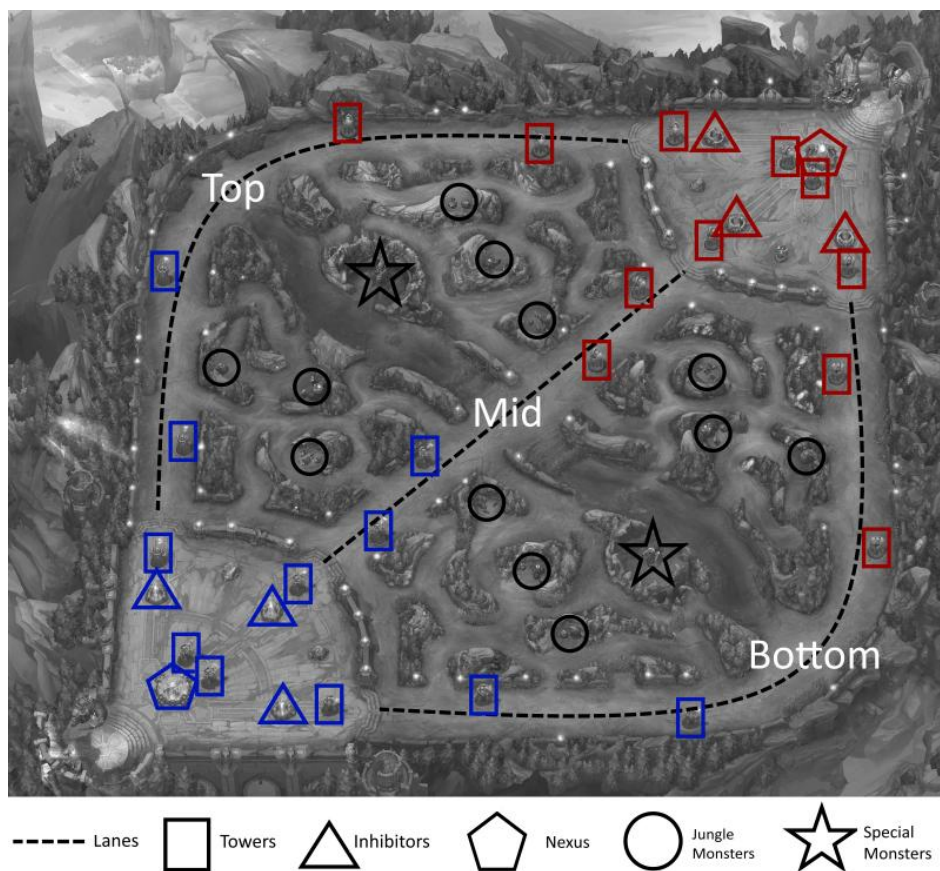
En conclusión, el presente trabajo permite vincular la formación técnica con la experiencia práctica en el dominio competitivo, con el fin de desarrollar una herramienta de valor que demuestre cómo la inteligencia artificial, sustentada en una base de conocimiento sólida y estructurada, puede constituirse como un aliado fundamental para la toma de decisiones en entornos de alta complejidad como los eSports.

1.2. Conceptos básicos y dominio del problema

Para comprender el funcionamiento del sistema experto desarrollado, resulta necesario exponer la dinámica de una partida de League of Legends y el proceso de traslación de dicho entorno a un lenguaje procesable por el sistema. El juego constituye un entorno de alta estrategia donde cada elección conlleva una consecuencia lógica. Se encuadra en el género MOBA (Multiplayer Online Battle Arena), donde dos equipos de cinco jugadores compiten en un mapa cerrado denominado "La Grieta del Invocador", con el objetivo final de destruir el nexo enemigo tras superar diversos obstáculos tácticos.

Aunque cada jugador ocupa una posición en el mapa, el sistema desarrollado se centra en el tipo de entidad o campeón que se maneja, mediante la creación de una ontología de etiquetas propia.

Figura 1. Mapa de la Grieta del Invocador con las líneas (Top, Mid, Bot y Jungla)



Fuente: League of Legends map and important locations, Afonso et al. (2019)

Como se ve en el mapa, cada jugador ocupa una posición. Sin embargo, lo que realmente importa para el sistema desarrollado no es solo dónde se coloca el jugador, sino qué "tipo" de entidad está manejando. Aquí es donde entra la parte técnica de este proyecto: la creación de una ontología de etiquetas propia.

1.2.1. Ontología de etiquetas y roles

A diferencia de las herramientas que emplean categorías estándar, se ha diseñado un vocabulario de etiquetas propio en la base de conocimientos para obtener una mayor precisión. Por ejemplo, el sistema permite identificar si un campeón ejerce como fuente de daño mágico (`magic_damage`) o si posee una alta capacidad de control de masas (`cc_heavy`). La Tabla 1 detalla las etiquetas clave definidas para el razonamiento del motor de inferencia. En la siguiente tabla se detallan algunas de las etiquetas clave definidas en el archivo `champions.json` y que el motor de inferencia utiliza para razonar:

Tabla 1. Ontología de Etiquetas del Sistema Experto.

Etiqueta (Tag)	Significado Estratégico	Función en el Motor
Bruiser / Fighter	Luchador cuerpo a cuerpo.	Evalúa la resistencia en peleas largas.
Marksman	Tirador de daño constante.	Detecta la necesidad de protección o de armadura enemiga.
Healing_self	Autocuración propia.	Dispara la regla de recomendación de "anti-curación".
Cc_heavy	Control de masas múltiple.	Activa la recomendación de "tenacidad" o resistencia.

Magic_damage	Daño de tipo mágico.	Se usa para equilibrar el daño del equipo aliado (AP vs AD).
--------------	----------------------	--

Fuente: Elaboración propia basada en la arquitectura del motor de inferencia.

1.2.2. El “hecho” (fact) y la explicabilidad

La toma de decisiones requiere la conversión del estado del juego en hechos lógicos procesables. Para ello, se ha definido la clase Fact como el núcleo del sistema, la cual permite transformar la información del entorno en datos estructurados. Lo que dota de valor a esta clase es que no solo almacena el dato técnico (por ejemplo, la presencia de un enemigo con alta resistencia), sino que también registra su procedencia (source).

Este diseño es fundamental para la explicabilidad, permitiendo que el sistema no se limite a indicar una acción, sino que sea capaz de informar sobre los factores específicos que han motivado dicha recomendación. Como señalan Russell y Norvig (2021), un sistema inteligente resulta de mayor utilidad si posee la capacidad de justificar sus conclusiones. Gracias al atributo source, el sistema no indica simplemente la compra de un objeto determinado, sino que es capaz de informar, por ejemplo, que la recomendación se basa en que un campeón específico posee etiquetas de curación, aportando así una base lógica a la sugerencia.

1.2.3. La fase de selección (draft) y las reglas de producción

El momento donde el sistema realmente demuestra su utilidad es en el Draft. Como se ha indicado anteriormente, esta es la fase previa a la partida donde se eligen los personajes y se establecen las bases de la victoria. Es, en estado puro, un juego de "piedra, papel o tijera" pero elevado a una potencia de miles de variables. Como bien señalan Mora-Cantallops y Sicart (2018), el conocimiento profundo del "metagame" y la capacidad de entender las sinergias en esta fase es lo que marca la diferencia entre un jugador promedio y uno experto.

Para que el sistema pueda razonar sobre este escenario, se ha diseñado una interfaz en Flask que permite al usuario volcar el estado actual de la partida. En términos de ingeniería de inteligencia artificial, lo que estamos haciendo aquí es alimentar la memoria de trabajo del motor de inferencia con "hechos" reales.

Figura 2. Interfaz del recomendador pidiendo al usuario insertar los datos de los campeones de la partida actual.

Una vez que el usuario introduce los personajes que están presentes en la sesión, el programa no se limita a leer nombres, lo que hace es mapear esos campeones con las etiquetas de la base de conocimientos. Por ejemplo, si el usuario introduce a "Soraka", el sistema genera automáticamente un hecho (Fact) asociado a la etiqueta `healing_support`. A partir de ahí, el motor de inferencia procesa esta información teniendo en cuenta la posición que le falta al usuario, las carencias de su propio equipo y, sobre todo, las amenazas que presenta el equipo enemigo.

Figura 3. Interfaz del recomendador tras un usuario haber insertado los personajes de la partida.



Al procesar esta composición de ejemplo, el programa ejecuta sus reglas de producción de forma secuencial. El resultado no es una respuesta única y cerrada, sino una conclusión razonada. El sistema recomienda tres posibles opciones de campeón para seleccionar, pero lo más importante desde el punto de vista de la ingeniería es que justifica su decisión. El programa avisa al usuario en todo momento de qué reglas han saltado y por qué se proponen esas soluciones, cumpliendo así con el principio de explicabilidad que mencionaba en la introducción.

Figura 4. El programa da como resultado su conclusión tras realizar los cálculos y recomienda las mejores opciones.



Como se puede observar en la Figura 4, el sistema realiza un análisis heurístico y muestra qué reglas lógicas han disparado esas recomendaciones. En este caso concreto, las tres reglas principales que han entrado en juego han sido: Anti-Curaciones, Anti-Tanques y Resistencia Mágica.

A la derecha de cada regla, el programa utiliza el atributo source que he implementado en la clase Fact para explicar qué personajes han provocado esa alerta. Por ejemplo, si un personaje como Soraka está presente, el motor detecta su alta capacidad de curación y recomienda automáticamente objetos con la etiqueta antiheal, como el Morellonomicon.

De igual forma ocurre con la regla de Anti-Tanques. Si el sistema identifica personajes robustos como Olaf, recomienda objetos de penetración de armadura. En el ejemplo visual, sugiere el Recordatorio Letal como la opción óptima, ya que es un objeto "híbrido" en su utilidad: aporta la penetración necesaria contra tanques y, a la vez, aplica anti-curaciones, optimizando así el inventario del jugador.

Por último, es relevante destacar que el sistema es capaz de trabajar con información incompleta. Si dejamos la selección del tirador (ADC/Bot) vacía, el programa analiza el hueco restante y propone campeones específicos para esa posición. Para refinar este ranking de tres opciones, el motor combina la lógica de las reglas con un cálculo matemático que cruza el Win Rate individual, el enfrentamiento directo (matchup) contra el ADC enemigo y la sinergia con el resto de los aliados ya seleccionados. De esta forma, paso de una simple lista de personajes a una asesoría estratégica integral basada en evidencias lógicas y estadísticas.

1.3. Estructura del trabajo

Para cerrar esta introducción, quiero resumir brevemente cómo he organizado el resto de la memoria. He dividido el trabajo en seis capítulos que recorren todo el camino, desde que surge la idea y estudio el problema hasta que consigo una solución técnica que funciona. Esta es la hoja de ruta que voy a seguir:

En el Capítulo 2: Contexto y Estado del Arte, me centro en analizar qué se ha hecho hasta ahora en el mundo de la IA aplicada a videojuegos. Aquí es donde comparo mi propuesta de sistema experto con las herramientas que ya existen en el mercado y con otros estudios realizados respecto al mismo tema, intentando explicar por qué mi enfoque en la "explicabilidad" aporta algo nuevo.

En el Capítulo 3: Objetivos y Metodología, detallo qué quiero conseguir exactamente con este proyecto (objetivos SMART) y qué pasos voy a seguir para no perderme por el camino. Explico cómo he usado Python y Flask siguiendo un orden lógico de desarrollo de software.

En el Capítulo 4: Análisis y Diseño de la Solución, entro de lleno en la arquitectura del sistema. En este apartado detallo los requisitos funcionales y el diseño de los componentes clave, como la base de conocimientos y el motor de inferencia. Es aquí donde explico formalmente cómo he modelado los "hechos" y las "reglas" para que el programa sea capaz de procesar el conocimiento experto que he inyectado en el sistema.

En el Capítulo 5: Implementación y Pruebas, describo el proceso de desarrollo técnico y la integración de las distintas tecnologías. Detallo el uso de Python para la lógica de negocio, Flask para la capa web y la conexión con la API de Riot Games. Además, presento los protocolos de prueba y los resultados obtenidos al someter al recomendador a situaciones de partida reales, validando así su precisión.

Finalmente, en el Capítulo 6: Conclusiones y Trabajo Futuro, realizo una síntesis de los resultados y valoro el grado de cumplimiento de los objetivos planteados. Este capítulo sirve también para proponer posibles expansiones del proyecto, como la implementación de lógica difusa o la automatización de la captura de datos en tiempo real, dejando la puerta abierta a futuras evoluciones de la herramienta.

2. Contexto y Estado del Arte

En este capítulo voy a profundizar en el marco teórico y tecnológico que rodea a este proyecto. Para entender por qué es necesario un sistema experto en League of Legends, primero es fundamental analizar la magnitud del problema estratégico al que se enfrentan los jugadores y, posteriormente, revisar qué soluciones se han desarrollado hasta la fecha, tanto en el ámbito académico como en el comercial. Mi objetivo aquí es situar mi propuesta dentro de la evolución de la Inteligencia Artificial, justificando por qué he decidido apostar por un modelo basado en reglas en un momento donde el Machine Learning parece dominarlo todo.

2.1. Análisis del contexto estratégico y complejidad del dominio

Para cualquier observador externo, un videojuego puede parecer un entorno simple de estímulos y diversión. Sin embargo, desde el punto de vista de la ingeniería de software, League of Legends es un sistema de una complejidad técnica abrumadora. Nos encontramos ante un entorno de información imperfecta y un espacio de estados que crece exponencialmente con cada nueva variable que se añade al juego.

2.1.1. La magnitud del problema: El espacio de decisiones

La complejidad del draft no es una exageración subjetiva. Si consideramos que hay más de 170 campeones disponibles y que en cada partida se seleccionan 10 (sin contar los 10 bloqueos iniciales ya que cada persona en la fase de draft tiene opción a prohibir el uso de un personaje en la partida), el número de combinaciones posibles es astronómico. A esto hay que sumar que cada campeón tiene un set de habilidades único y estadísticas que cambian en cada parche. Según Hodge et al. (2017), los juegos de tipo MOBA presentan retos de optimización y predicción que superan en muchas dimensiones a juegos clásicos, debido a la naturaleza dinámica de los datos y a que la ejecución se realiza en tiempo real con información incompleta.

He observado que esta saturación de información genera lo que en psicología cognitiva se conoce como "carga cognitiva excesiva". El jugador, bajo la presión del cronómetro en la fase de selección, no puede procesar todos los enfrentamientos, sinergias y debilidades de forma puramente racional. Es en este punto donde he identificado la necesidad de una herramienta que actúe como un filtro inteligente, transformando miles de datos brutos en recomendaciones digeribles que permitan al usuario tomar la mejor decisión posible sin verse desbordado por la complejidad del sistema.

2.2. Estado del arte: soluciones actuales y su evolución

Para situar mi proyecto en el panorama tecnológico actual, es necesario analizar qué herramientas tienen a su disposición los jugadores hoy en día y cómo ha evolucionado la investigación académica en este campo. A grandes rasgos, podemos dividir las soluciones existentes en dos categorías, las plataformas comerciales basadas en analítica de datos masivos y los estudios académicos centrados en modelos predictivos de inteligencia artificial.

2.2.1. Análisis de herramientas comerciales: El predominio de la estadística descriptiva

En el ecosistema competitivo de League of Legends, han proliferado herramientas muy populares como OP.GG, Blitz.gg, Porofessor, Mobalytics o U.GG. Tras analizar el funcionamiento de estas plataformas, he identificado que su principal fortaleza es, al mismo tiempo, su mayor limitación: se basan casi exclusivamente en la analítica descriptiva.

Estas herramientas recolectan millones de datos de partidas a través de la API de Riot Games para mostrar tendencias. Por ejemplo, pueden indicarte que un campeón tiene un 53% de win rate o que un objeto es el más frecuente en una determinada posición. Sin embargo, como mencionan Russell y Norvig (2021) en su análisis sobre la utilidad de los sistemas inteligentes, la información sin contexto tiene un valor limitado para la toma de decisiones complejas. El problema de estas aplicaciones es que funcionan como "cajas negras", te dicen qué elegir, pero no te explican por qué esa elección es buena contra tu rival específico o cómo encaja en la sinergia de tu equipo.

Lo que busco con mi sistema experto es alejarme de esa estadística fría. Mientras que Blitz.gg te recomienda un objeto porque "mucha gente lo compra", mi motor de inferencia te lo recomienda porque ha detectado un "hecho" lógico (por ejemplo, el enemigo tiene la etiqueta `healing_self`) y ha aplicado una regla de conocimiento experto.

En definitiva, aunque las herramientas comerciales actuales ofrecen datos estadísticos precisos, su naturaleza de "caja negra" genera una brecha en la comprensión estratégica del

usuario. Esta limitación justifica el desarrollo de un sistema basado en reglas que trascienda la estadística descriptiva para ofrecer una asesoría justificada.

2.2.2. Evolución de la IA en los eSports: De la predicción a la explicación

En el ámbito académico, la investigación ha avanzado significativamente hacia la predicción de resultados. Un estudio fundamental en este campo es el de Hodge et al. (2017), quienes exploraron cómo algoritmos de Machine Learning pueden predecir quién ganará una partida de un MOBA analizando las variables del draft. Estos modelos son extremadamente precisos, pero suelen requerir una capacidad computacional enorme y son difíciles de interpretar para un usuario que no sea experto en datos.

Recientemente, el foco de la comunidad científica ha empezado a girar hacia la Inteligencia Artificial Explicable (XAI). Según Adadi y Berrada (2018), el valor real de una IA en entornos donde hay un humano tomando la decisión final (como es el caso de un jugador en plena fase de selección) reside en su capacidad para justificar sus conclusiones. No basta con que el sistema sea un oráculo que acierta, el usuario necesita entender el razonamiento para confiar en la herramienta y, de paso, mejorar su propio criterio estratégico. Es precisamente en esta corriente de la XAI donde se encuadra mi propuesta de Sistema Experto.

2.2.3. El sistema experto como alternativa pedagógica

A diferencia de los modelos de aprendizaje profundo (Deep Learning) que se han usado en proyectos famosos como OpenAI Five para Dota 2, yo he apostado por un sistema basado en reglas de producción. La razón es sencilla, la flexibilidad y la pedagogía.

En un entorno tan cambiante como el de League of Legends, donde cada dos semanas hay un parche que cambia las reglas del juego, un sistema experto es mucho más ágil. No necesito reentrenar un modelo con millones de nuevas partidas, simplemente puedo actualizar la base de conocimientos o ajustar una regla de prioridad. Esta capacidad de inyectar conocimiento experto de forma inmediata es lo que permite que mi solución sea más que un simple

recomendador, es un consultor lógico que ayuda al jugador a comprender la capa interna estratégica del juego.

A diferencia de los enfoques basados exclusivamente en Machine Learning, el sistema experto propuesto permite una actualización inmediata del conocimiento ante cambios en el metagame. Mientras que el aprendizaje profundo destaca en la predicción, este enfoque prioriza la transparencia lógica, facilitando que el usuario comprenda la capa estratégica interna del juego.

2.2.4. Evaluación crítica y matriz comparativa de enfoques tecnológicos

Con el objetivo de justificar la elección metodológica y tecnológica de este proyecto, resulta imprescindible realizar un análisis comparativo estructurado que vaya más allá de la mera descripción de las herramientas vigentes. Las soluciones identificadas en el estado del arte se evalúan bajo cuatro criterios de ingeniería de software esenciales para el entorno del draft competitivo de League of Legends: la transparencia del razonamiento (explicabilidad), la agilidad de actualización ante la volatilidad del dominio (mantenimiento), la infraestructura de datos requerida y el nivel de carga cognitiva impuesto al usuario bajo presión de tiempo.

Tabla 2. Matriz comparativa de enfoques tecnológicos para la asistencia en el draft.

Criterio Técnico	Plataformas de Análítica Descriptiva	Modelos Predictivos de Machine Learning Puro	Solución Propuesta: Sistema Experto Híbrido con XAI
Naturaleza del Enfoque	Agregación estadística descriptiva masiva.	Modelos matemáticos de caja negra (Redes Neuronales).	Razonamiento lógico- simbólico acoplado a estadística contextual.
Transparencia (Explicabilidad)	Nula. Muestra datos masivos de frecuencia	Nula o muy compleja. Funciona	Alta. Traza de razonamiento explícita

	sin justificación causal.	como un oráculo probabilístico.	en lenguaje natural mediante el atributo source.
Mantenimiento y Volatilidad	Pasivo. Depende de la acumulación de millones de partidas tras cada parche.	Costoso. Requiere el reentrenamiento completo del modelo ante cambios del metagame.	Inmediato. Flexibilidad para modificar o priorizar reglas lógicas de producción en JSON.
Carga Cognitiva del Usuario	Alta. El jugador debe interpretar tablas complejas de porcentajes en segundos.	Baja. Proporciona un porcentaje de victoria final directo sin contexto.	Baja-Pedagógica. Ofrece una recomendación jerarquizada junto con su argumento táctico.

Fuente: Elaboración propia basada en la revisión crítica de la literatura y soluciones actuales.

Como se evidencia en la matriz comparativa de la Tabla 2, las plataformas comerciales dominantes en el mercado actual han optado por un enfoque de fuerza bruta basado en la ingesta masiva de datos a través de la API oficial de Riot Games. Herramientas analíticas descriptivas como Blitz.gg o Porofessor realizan una excelente labor de minería de datos, pero sufren de un vacío funcional crítico: la ausencia de causalidad. Un jugador competitivo en la fase de selección no necesita únicamente saber que un campeón ostenta un determinado porcentaje de victorias global; necesita comprender bajo qué condiciones estructurales de la composición (aliada o enemiga) se sostiene dicho indicador. Al carecer de un motor de inferencia capaz de asociar variables lógicas, estas herramientas imponen una elevada carga cognitiva, obligando al usuario a correlacionar mentalmente y en pocos segundos múltiples tablas de estadísticas masivas, lo que incrementa el riesgo de tomar una elección subóptima debido a la presión del cronómetro del draft.

Por otro lado, la literatura científica contemporánea ha concentrado sus esfuerzos en el despliegue de clasificadores predictivos basados en redes neuronales o algoritmos de Machine Learning tradicional para vaticinar el éxito de una composición (Hodge et al., 2017). Si bien estos enfoques logran precisiones matemáticas sobresalientes, su viabilidad práctica como asistentes pedagógicos se ve severamente limitada en la ingeniería del software por la opacidad algorítmica. Estos modelos determinan una probabilidad de victoria operando sobre vectores densos de características numéricas que actúan como una "caja negra". Para un jugador humano, el resultado emitido carece de una justificación táctica accionable, puesto que no existe un flujo explicativo que desglose el porqué estratégico de la recomendación. A esto se suma el alto coste de reentrenamiento ante la volatilidad del entorno, al ser League of Legends un dominio regulado por parches de balanceo bisemanales, cualquier alteración menor en los atributos de un campeón puede invalidar los patrones estadísticos previamente aprendidos por el modelo, forzando ciclos constantes de cómputo masivo y denotando una gran rigidez ante cambios rápidos en el "metagame".

La arquitectura diseñada en este trabajo resuelve de manera elegante las deficiencias de los enfoques anteriores mediante un paradigma híbrido que sitúa la Inteligencia Artificial Explicable (XAI) en el centro de la solución. En lugar de competir con la precisión probabilística pura, el sistema utiliza un motor de inferencia lógico-simbólico que emula el proceso cognitivo de un analista humano experto. La inyección de conocimiento declarativo mediante una ontología parametrizada en archivos JSON descentraliza la inteligencia de la aplicación. Si un parche altera radicalmente las mecánicas del juego, el sistema no requiere un reentrenamiento masivo, basta con una actualización atómica en las etiquetas del archivo JSON o un ajuste en la jerarquía de prioridades del motor para restablecer la precisión estratégica del recomendador. Asimismo, la inclusión del atributo source dentro de la clase Fact transforma la experiencia del usuario, permitiendo generar argumentos explícitos en lenguaje natural. De este modo, la solución propuesta no solo optimiza la toma de decisiones técnicas bajo presión, sino que dota al proyecto del rigor científico exigido por las corrientes actuales de la IA explicable.

3. Objetivos y metodología de trabajo

Tras haber analizado en los capítulos anteriores la enorme complejidad técnica que rodea a League of Legends y comprobar cómo las soluciones actuales suelen dejar al usuario a ciegas respecto a la lógica de sus consejos, en este apartado voy a concretar qué es lo que me he propuesto lograr con este proyecto. No se trata solo de programar una herramienta que funcione, sino de seguir un proceso estructurado que garantice que el sistema experto es fiable, escalable y, sobre todo, útil para un jugador que busca mejorar su capacidad estratégica.

En las siguientes páginas, voy a desglosar mi objetivo general en varias metas más pequeñas y manejables, lo que me permitirá avanzar con paso firme durante el desarrollo. Además, describiré la metodología de trabajo que he seguido, un camino que combina la disciplina de la ingeniería de software con la flexibilidad necesaria para adaptar el conocimiento de un analista humano a un motor de reglas en Python. Como bien indica Sommerville (2011), cualquier proyecto de software exitoso debe basarse en un proceso bien definido que permita gestionar la complejidad, y en mi caso, ese proceso ha sido fundamental para no perderme entre los más de 170 campeones y los miles de interacciones posibles del juego.

3.1. Objetivo general

El objetivo principal de este trabajo es diseñar e implementar un sistema experto basado en reglas que asista a los jugadores de League of Legends durante la fase de draft, proporcionando recomendaciones estratégicas de personajes y equipamiento que sean totalmente explicables y fáciles de comprender para el usuario, haciendo que este no solo tome la mejor decisión táctica a nivel de número en porcentaje de victorias, sino que también llegue a entender el motivo detrás de esta decisión.

Con este proyecto, no busco simplemente crear otro recomendador estadístico basado en números brutos, sino desarrollar una herramienta que sea capaz de razonar sus propuestas. La idea es que el sistema actúe como un consultor táctico, traduciendo el conocimiento

experto sobre sinergias y debilidades del juego en consejos directos. Para lograrlo, he apostado por construir un motor de inferencia propio en Python y una interfaz web accesible mediante Flask, permitiendo que cualquier jugador pueda recibir una asesoría estratégica de alto nivel en tiempo real.

3.2. Objetivos específicos

Para alcanzar la meta principal que acabo de describir, he desglosado el trabajo en los siguientes objetivos operativos:

- Diseñar una base de conocimientos estructurada mediante una ontología de etiquetas que categorice los roles y utilidades de los campeones.
- Desarrollar un motor de inferencia en Python capaz de procesar reglas de producción tipo "IF-THEN" sobre la memoria de trabajo de la partida.
- Implementar un sistema de explicabilidad (XAI) que muestre la traza del razonamiento y justifique las sugerencias al usuario.
- Construir una interfaz web funcional con Flask que facilite la interacción intuitiva en tiempo real.
- Validar la precisión del sistema mediante escenarios reales para comprobar la eficacia de las reglas de sinergia y contra-elección.

3.3. Metodología de trabajo

3.3.1. Enfoque Ágil: Desarrollo Iterativo e Incremental

El diseño y construcción de un sistema experto orientado a un entorno altamente volátil como League of Legends desaconseja el uso de metodologías tradicionales o en cascada (Waterfall). Al tratarse de un dominio sujeto a parches de balanceo continuos, los requisitos del conocimiento estratégico pueden variar durante el propio ciclo de desarrollo. Por este motivo, para la ejecución de este proyecto se ha adoptado un paradigma ágil basado en un enfoque de desarrollo iterativo e incremental.

Siguiendo los principios de la ingeniería de software moderna (Sommerville, 2011), este enfoque divide el ciclo de vida del proyecto en iteraciones cortas o sprints. En lugar de intentar construir el motor de inferencia completo en una única fase monolítica, el sistema se ha desarrollado mediante módulos funcionales que añaden valor de forma progresiva. Cada iteración abarca las tareas de extracción de conocimiento, codificación de reglas lógicas, diseño de interfaz y pruebas de validación.

La principal ventaja metodológica de este diseño radica en la retroalimentación temprana. El desarrollo incremental ha permitido construir primero un prototipo básico por consola que validaba las reglas de encadenamiento hacia adelante, para posteriormente iterar sobre él añadiendo la capa web en Flask, la integración estadística con SQLite y, finalmente, el módulo de Inteligencia Artificial Explicable (XAI). Este ciclo de refactorización continua asegura que cualquier desviación en la lógica de priorización del motor sea detectada y corregida rápidamente mediante validaciones heurísticas, garantizando que el producto final cumpla estrictamente con los requisitos funcionales y no funcionales planteados.

Para estructurar operativamente este enfoque ágil, el trabajo se ha dividido en cuatro fases principales que se retroalimentan entre sí:

- **1. Ingeniería del conocimiento y extracción de datos**

La primera etapa consistió en definir de dónde vendría la inteligencia del sistema. Se realizó una investigación profunda sobre el metagame actual y se utilizó la API de Riot Games (DataDragon) para obtener los datos base. El reto en esta fase no fue solo descargar información, sino transformarla en algo útil, se creó una ontología de etiquetas propia en un archivo JSON que permitiera al sistema clasificar a los personajes por su comportamiento real en la partida (como healing_self o cc_heavy) y no solo por su nombre.

- **2. Diseño y desarrollo del motor de inferencia**

Con los datos ya estructurados, se procedió a programar el cerebro de la aplicación en Python. Siguiendo los principios de los sistemas expertos, se diseñó la clase Fact para gestionar la memoria de trabajo y la clase Engine para procesar las reglas de producción. En esta fase, el trabajo fue puramente de lógica, traducir el conocimiento estratégico de un jugador experto a reglas de programación tipo IF-THEN.

- **3. Integración de la Interfaz Web y Experiencia de Usuario**

Una vez que el motor era capaz de dar recomendaciones correctas por consola, se desarrolló la capa visual utilizando el framework Flask. El objetivo aquí fue crear una interfaz que fuera rápida de usar (crucial durante un draft real) y que permitiera mostrar visualmente la traza del razonamiento. Esta fase incluyó la gestión de recursos estáticos, como las imágenes de los campeones.

- **4. Pruebas, Validación y Refinamiento**

La última fase ha sido un proceso constante de "prueba y error" utilizando casos reales de partidas clasificatorias y de la escena profesional. He sometido al sistema a composiciones de equipo variadas para verificar si las recomendaciones eran coherentes o se ajustaban a casos reales. Esto me permitió ajustar las prioridades de las reglas (por ejemplo, dando más peso a la falta de daño mágico o a la necesidad de cortar curaciones enemigas) hasta lograr un equilibrio que aporte un valor estratégico real al usuario.

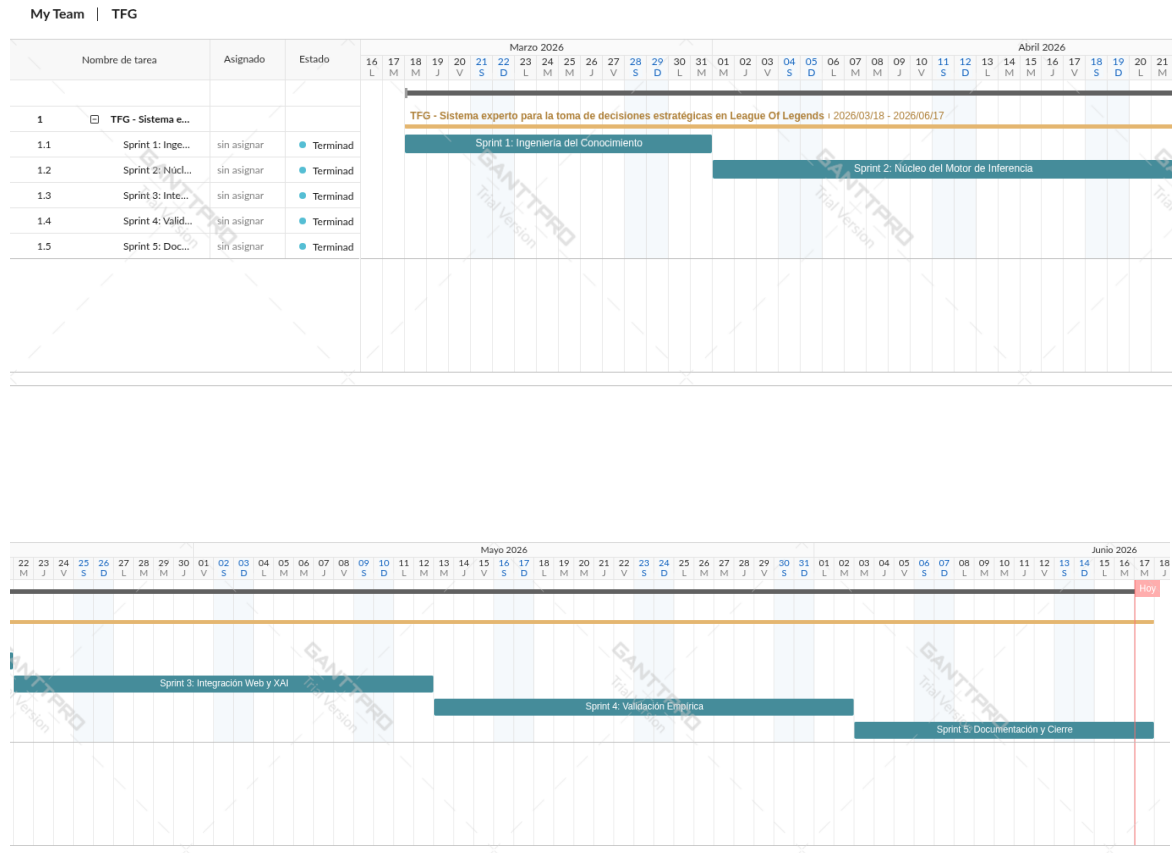
3.3.2. Cronograma de Trabajo y Diagrama de Gantt

Para garantizar el cumplimiento de los plazos de entrega y alinear el progreso con el enfoque ágil iterativo (Scrum adaptado), el periodo de desarrollo de tres meses (desde el 18 de marzo hasta el 17 de junio de 2026) se estructuró en cinco iteraciones o Sprints. Cada Sprint fue diseñado para culminar con un entregable funcional o documental evaluable.

A continuación, se detalla la planificación temporal de las fases descritas anteriormente:

- **Sprint 1:** Ingeniería del Conocimiento (18/03/2026 - 31/03/2026): Recopilación de datos del metagame, conexión inicial con la API de Riot Games y definición de la ontología de etiquetas en formato JSON.
- **Sprint 2:** Núcleo del Motor de Inferencia (01/04/2026 - 21/04/2026): Programación en Python de las clases Fact y Engine. Implementación del algoritmo de forward chaining y establecimiento de las jerarquías de prioridad para la resolución de conflictos.
- **Sprint 3:** Integración Web y XAI (22/04/2026 - 12/05/2026): Desarrollo del backend con Flask, mapeo ontológico en SQLite y creación de la interfaz de usuario. Implementación del atributo source para la explicabilidad (XAI).
- **Sprint 4:** Validación Empírica (13/05/2026 - 02/06/2026): Fase de pruebas exhaustivas. Simulación de escenarios de draft competitivos reales y ajuste de los pesos del algoritmo de scoring híbrido (70/30).
- **Sprint 5:** Documentación y Cierre (03/06/2026 - 17/06/2026): Redacción final de la memoria académica, estructuración de anexos, revisión de formato APA y preparación del predepósito del trabajo.

Figura 5. Diagrama de Gantt reflejando los Sprints de desarrollo del proyecto.



Fuente: Elaboración propia mediante la herramienta GanttPRO.

3.3.3. Herramientas de Gestión del Proyecto

Para dar soporte a la metodología iterativa y asegurar la integridad del código fuente, se hizo uso de herramientas estándar de la industria del software.

El control de versiones se gestionó de manera centralizada mediante Git, utilizando un repositorio remoto en GitHub que permite la trazabilidad de todos los cambios realizados en el motor de inferencia y la interfaz web. El desarrollo del código se llevó a cabo en el entorno de desarrollo integrado (IDE) Visual Studio Code, aprovechando sus extensiones para la depuración de Python y la manipulación de bases de datos SQLite en tiempo real. Finalmente, para la gestión documental y la redacción de la memoria académica, se emplearon herramientas ofimáticas asegurando el cumplimiento estricto de la normativa de estilo APA y las directrices de la Universidad Internacional de La Rioja.

4. Análisis y Diseño de la Solución

En este capítulo se profundiza en la arquitectura técnica y el diseño detallado del sistema experto propuesto. Tras haber definido los objetivos y la metodología en las secciones anteriores, resulta imperativo detallar cómo se transforman los requisitos funcionales en una solución tecnológica robusta y escalable. El diseño de la solución se fundamenta en la separación de responsabilidades, diferenciando claramente entre la capa de adquisición de datos (API de Riot Games), el núcleo de razonamiento (motor de inferencia en Python) y la capa de presentación (interfaz web en Flask).

El análisis aquí presentado no solo cubre los aspectos estáticos del software, como el modelo de datos y la estructura de las clases, sino también el comportamiento dinámico del sistema ante la variabilidad del draft. Se presta especial atención a la arquitectura orientada a la explicabilidad, asegurando que cada componente contribuya a la transparencia del razonamiento estratégico, cumpliendo así con el pilar fundamental de la Inteligencia Artificial Explicable (XAI) definido en el marco teórico.

4.1. Análisis de requisitos

El análisis de requisitos constituye la fase fundamental para garantizar que el sistema experto desarrollado no solo sea técnicamente funcional, sino que también responda de manera efectiva a las necesidades estratégicas de los usuarios en un entorno competitivo. En esta sección se identifican y detallan las capacidades que el sistema debe poseer (requisitos funcionales) y las restricciones bajo las cuales debe operar (requisitos no funcionales).

4.1.1. Identificación de actores

En el contexto de este sistema experto, se identifica un actor principal:

- **Usuario (Jugador/Analista):** Es el actor que interactúa con la interfaz web durante la fase de draft. Su función es alimentar el sistema con los datos de la partida actual

(campeones aliados y enemigos) y consumir las recomendaciones justificadas que genera el motor de inferencia.

4.1.2. Requisitos funcionales (RF)

Los requisitos funcionales definen los servicios que el sistema debe proporcionar. Para este proyecto, se han categorizado según su área de impacto:

- **GESTIÓN DE DATOS Y CONOCIMIENTO**
 - **RF-01:** Integración con la API de Riot Games. El sistema debe ser capaz de conectarse a los servicios de Data Dragon y las APIs de Riot para extraer información actualizada sobre campeones, objetos y estadísticas de win rate.
 - **RF-02:** Definición de Ontología Estratégica. El sistema debe implementar una estructura de conocimiento (archivo JSON) que categorice a los campeones mediante etiquetas propias (tags) que representen fielmente su rol en el metagame actual.
 - **RF-03:** Gestión de la Memoria de Trabajo. El sistema debe permitir la creación y manipulación de hechos (Facts) que representen el estado actual de la partida, almacenando tanto el dato estratégico como su origen (source).
- **LÓGICA DE INFERENCIA**
 - **RF-04:** Ejecución del Motor de Inferencia. El sistema debe procesar las reglas de producción mediante un algoritmo de encadenamiento hacia adelante (forward chaining) hasta alcanzar un punto fijo de razonamiento.
 - **RF-05:** Resolución de Conflictos y Priorización. Ante la activación simultánea de múltiples reglas, el motor debe aplicar una jerarquía de prioridades estática y dinámica para determinar el orden de disparo más efectivo estratégicamente.

- **RF-06:** Generación de Ranking Híbrido. El sistema debe ser capaz de combinar las deducciones lógicas del motor con los datos estadísticos brutos para generar una lista ordenada de recomendaciones de campeones y objetos.
- EXPLICABILIDAD E INTERFAZ
 - **RF-07:** Sistema de Explicabilidad (XAI). El sistema debe generar una traza de razonamiento en lenguaje natural que justifique cada recomendación basándose en los hechos presentes en la memoria de trabajo.
 - **RF-08:** Interfaz Web de Draft. Se debe proporcionar una interfaz intuitiva que permita la selección visual de campeones y muestre de forma clara las conclusiones del cálculo estratégico.

4.1.3. Requisitos no funcionales (RNF)

A diferencia de los funcionales, los requisitos no funcionales determinan la calidad y la viabilidad técnica del sistema en un entorno de producción real. Se detallan a continuación con su correspondiente justificación técnica:

- **RNF-01:** Eficiencia y Tiempo de Respuesta: El sistema debe procesar la lógica de inferencia y devolver los resultados en un tiempo inferior a 2 segundos. Justificación: En la fase de draft de League of Legends, los jugadores disponen de aproximadamente 30 segundos para confirmar su elección. Un retraso en el cálculo degradaría la utilidad de la herramienta, imposibilitando la aplicación de la estrategia recomendada.
- **RNF-02:** Robustez y Disponibilidad: El sistema debe implementar mecanismos de failover o degradación elegante ante la indisponibilidad de las APIs externas de Riot Games. Justificación: Si la API de win rates falla, el motor de inferencia debe ser capaz de operar utilizando exclusivamente la base de conocimientos estática en JSON para no interrumpir el servicio al usuario.

- **RNF-03:** Usabilidad y Carga Cognitiva: La interfaz desarrollada en Flask debe minimizar el número de interacciones necesarias para obtener una recomendación. Justificación: El usuario se encuentra bajo una situación de alta presión competitiva; por tanto, la jerarquía visual debe priorizar la "explicación de la regla" y el "campeón recomendado" sobre el resto de los datos secundarios.
- **RNF-04:** Portabilidad y Despliegue: Al estar basado en Python y Flask, el sistema debe ser capaz de ejecutarse en cualquier entorno que soporte un servidor WSGI, asegurando la compatibilidad multiplataforma.

4.1.4. Análisis de casos de uso

El análisis de casos de uso permite describir las interacciones entre el actor (Usuario) y el sistema para alcanzar un objetivo específico. A continuación, se presenta el diagrama de casos de uso y la descripción detallada de los procesos críticos.

1. CU-01: Consultar Recomendación de Draft

Este es el proceso central del sistema, donde se pone en marcha el motor de inferencia para generar una asesoría estratégica.

Tabla 3. Especificación del Caso de Uso CU-01: Consultar Recomendación de Draft

Campo	Descripción
Actor	Usuario (Jugador / Analista)
Precondición	El sistema debe haber cargado la base de conocimientos JSON y la conexión con la API de Riot debe estar activa.

Flujo Principal	1. El usuario accede a la interfaz de draft.
	2. El usuario selecciona los campeones enemigos detectados.
	3. El usuario solicita el análisis de la composición.
	4. El sistema mapea los campeones con sus etiquetas ontológicas.
	5. El motor de inferencia dispara las reglas de producción pertinentes.
	6. Se realiza el cálculo de scoring híbrido (Lógica + Estadística).
	7. El sistema muestra los 3 mejores campeones y los objetos recomendados con su respectiva justificación
Postcondición	Se muestra la traza de razonamiento (XAI) en pantalla.

Fuente: Elaboración propia basada en el análisis de requisitos del sistema.

2. CU-02: Sincronización de Datos Estadísticos

Garantiza que las recomendaciones basadas en win rate sean actuales y coherentes con el parche vigente del juego.

Tabla 4. Especificación del Caso de Uso CU-02: Sincronización de Datos Estadísticos.

Campo	Descripción
Actor	Sistema (Automático) / Administrador
Precondición	Disponer de una API Key de Riot Games válida.
Flujo Principal	1. El sistema realiza una petición a los servicios de Data Dragon. 2. Se descarga los últimos cambios en estadísticas base de los campeones. 3. El sistema actualiza la base de datos SQLite con los nuevos porcentajes de victoria y matchups.
Postcondición	Los cálculos de scoring del motor de inferencia utilizan datos del parche actual.

Fuente: Elaboración propia basada en el análisis de requisitos del sistema.

4.1.5. Análisis de restricciones del sistema

Es imperativo considerar las limitaciones técnicas que enmarcan el diseño de la solución:

1. Límite de peticiones de la API (Rate Limiting): Las APIs de Riot Games imponen restricciones de tráfico (ej. 20 peticiones cada 1 segundo). Esto obliga al sistema a implementar una arquitectura de caché o almacenamiento local en SQLite para no exceder dichos límites durante consultas masivas.
2. Volatilidad del Dominio: El metagame de League of Legends cambia cada dos semanas. Esta restricción justifica la elección de un sistema experto, ya que permite modificar la base de conocimientos sin alterar el código fuente del motor de inferencia.

4.1.6. Matriz de trazabilidad de requisitos

Para asegurar la coherencia del diseño, se presenta la siguiente matriz que vincula los requisitos funcionales con los objetivos específicos definidos en el Capítulo 3:

Tabla 5. Matriz de Trazabilidad entre Requisitos Funcionales y Objetivos del Proyecto.

ID Requisitos	Descripción Breve del Requisito	Objetivo Vinculado
RF-01	Integración con la API Riot Games.	Objetivo 2 y 5: Desarrollo del motor / Validación.
RF-02	Definición de Ontología Estratégica.	Objetivo 1: Diseñar base de conocimientos.
RF-03	Gestión de la Memoria de Trabajo (Facts).	Objetivo 2: Desarrollo del motor de inferencia.
RF-04	Ejecución del Motor (Forward Chaining).	Objetivo 2: Desarrollo del motor de inferencia.
RF-05	Resolución de Conflictos y Priorización.	Objetivo 2: Desarrollo del motor de inferencia.

RF-06	Generación de Ranking Híbrido.	Objetivo 2: Desarrollo del motor de inferencia.
RF-07	Sistema de Explicabilidad (XAI).	Objetivo 3: Implementar sistema de explicabilidad.
RF-08	Interfaz Web de Draft (Flask).	Objetivo 4: Construir interfaz web funcional.

Fuente: Elaboración propia basada en la planificación técnica del proyecto.

4.2. Arquitectura del sistema

El diseño arquitectónico de la solución se fundamenta en un modelo de capas que garantiza la independencia entre la lógica de negocio, la gestión de datos y la interfaz de usuario. Esta estructura se basa en el principio de Separación de Responsabilidades (Separation of Concerns), lo que permite que el motor de inferencia opere de manera aislada, recibiendo hechos y devolviendo conclusiones sin depender de la implementación de la capa visual. Esta arquitectura facilita el mantenimiento evolutivo, permitiendo actualizar la lógica estratégica sin alterar la infraestructura del servidor.

4.2.1. Vista General de la Arquitectura

La arquitectura se organiza siguiendo un patrón que facilita la comunicación bidireccional entre el usuario y el núcleo del sistema. Se ha optado por un enfoque basado en servicios donde el servidor Flask actúa como orquestador central. El flujo de información es estrictamente unidireccional en el procesamiento: se inicia en la capa de presentación, se valida en el servidor de aplicaciones y se apoya en una capa de persistencia híbrida.

A continuación, se describen los tres pilares técnicos que sostienen la solución:

1. **Capa de Presentación (Frontend):** Se ha desarrollado mediante el motor de plantillas Jinja2 integrado en Flask. Su función principal es la captura de parámetros del draft mediante formularios dinámicos y la representación visual de los resultados. Esta capa es la encargada de transformar los objetos JSON devueltos por el motor en trazas de Inteligencia Artificial Explicable (XAI) legibles por el ser humano.
2. **Capa de Lógica (Backend):** Constituye el "cerebro" del sistema. Aquí reside el motor de inferencia desarrollado íntegramente en Python, que implementa un algoritmo de encadenamiento hacia adelante (forward chaining). Esta capa recibe los datos de los campeones, los mapea contra la base de conocimientos y dispara las reglas de producción que generan el conocimiento derivado.
3. **Capa de Datos:** Implementa una estrategia de persistencia mixta. Por un lado, utiliza una base de conocimientos estática en archivos JSON para la ontología de etiquetas. Por otro, emplea una base de datos SQLite para el almacenamiento local de estadísticas masivas (win rates y matchups) obtenidas de la API de Riot Games, optimizando así los tiempos de consulta.

4.2.2. Diseño de Componentes del Backend

El backend se ha diseñado siguiendo criterios de modularidad y bajo acoplamiento. La elección de Python como lenguaje vehicular se justifica por su robustez en el procesamiento de lógica simbólica y la facilidad de integración con bibliotecas de análisis de datos.

- **Servidor Flask:** Este componente actúa como controlador central del patrón arquitectónico. Gestiona el enrutamiento de peticiones HTTP, la validación de los datos de entrada del draft y la coordinación de las llamadas al motor de inferencia. Su diseño ligero permite una ejecución rápida, cumpliendo con el requisito de tiempo de respuesta crítico.

- **Módulo de Inferencia (Inference Engine):** Es el componente de mayor complejidad técnica. Su función es procesar la Memoria de Trabajo comparando los hechos actuales contra el conjunto de reglas estratégicas. Al estar desacoplado de la interfaz, el motor puede ser testeado de forma independiente mediante pruebas unitarias, garantizando la fiabilidad de las recomendaciones.
- **Gestor de Explicabilidad:** Es el componente encargado de la trazabilidad. Recupera el atributo source de cada hecho que ha provocado el disparo de una regla para construir la justificación lógica. Este componente asegura que el sistema no actúe como una "caja negra", aportando transparencia a cada sugerencia estratégica.

4.2.3. Integración con Servicios Externos y Almacenamiento Local

La robustez del sistema experto está vinculada a la fidelidad de sus datos. La integración con la API de Riot Games plantea desafíos de latencia y restricciones de tráfico que se han resuelto mediante el siguiente diseño:

- **Sincronizador de Data Dragon:** Este módulo automatiza la actualización de los metadatos de los campeones. Cada vez que Riot Games publica un nuevo parche, el sincronizador descarga las últimas versiones de las imágenes y archivos de datos, asegurando que la ontología del sistema experto sea coherente con el estado actual del juego.
- **Caché de Estadísticas en SQLite:** Para cumplir con el requisito de respuesta en menos de 2 segundos, el sistema evita realizar peticiones directas a la API durante la fase de draft. En su lugar, consulta una base de datos local previamente sincronizada. Este mecanismo de caché actúa como salvaguarda ante posibles caídas del servicio externo (failover).
- **Mapeador Ontológico:** Funciona como un traductor entre el mundo estadístico y el mundo lógico. Transforma los IDs numéricos de la API en etiquetas ontológicas (como

cc_heavy o magic_damage), permitiendo que el motor de inferencia razone sobre conceptos estratégicos en lugar de simples identificadores de base de datos.

4.2.4. Ciclo de Vida de una Petición de Recomendación

Para comprender la dinámica del sistema, es necesario detallar el flujo de operaciones desde que el usuario interactúa con la interfaz:

1. **Captura:** El usuario selecciona la composición en la interfaz web.
2. **Transmisión:** Flask recibe los datos y los envía al Mapeador Ontológico.
3. **Inferencia:** El motor carga las etiquetas, genera los hechos iniciales y dispara las reglas de producción.
4. **Enriquecimiento:** Se consultan los win rates en la base de datos SQLite para realizar el cálculo del ranking híbrido.
5. **Explicación:** El Gestor de Explicabilidad recopila las fuentes de los hechos disparados.
6. **Respuesta:** El servidor renderiza la plantilla con los resultados y las justificaciones, devolviendo la información al navegador.

4.3. Diseño del Modelo de Datos

El éxito de un sistema experto reside en la calidad y estructura de su base de conocimientos. En este proyecto, el modelo de datos se ha diseñado para ser híbrido, combinando la flexibilidad de los archivos JSON para la lógica simbólica y la eficiencia de SQLite para los datos estadísticos masivos. A continuación, se detalla la implementación técnica de las estructuras que permiten el razonamiento del sistema.

4.3.1. Estructura de la Ontología de Conocimiento (JSON)

La inteligencia del sistema se almacena en un archivo denominado champions.json, que actúa como la base de conocimientos declarativa. Se ha optado por el formato JSON debido a su facilidad para representar estructuras jerárquicas y su perfecta integración con los diccionarios de Python.

Cada entrada en la ontología representa un campeón y contiene un conjunto de atributos estratégicos que el motor de inferencia utiliza como premisas para las reglas.

Tabla 6. Especificación de campos en la ontología de campeones.

Campo	Tipo	Descripción Estratégica
Id	String	Identificador único del campeón (coincidente con la API de Riot).
name	String	Nombre legible del personaje.
tags	List	Etiquetas ontológicas que definen el rol (ej. tank, cc_heavy, healing_self).
role	String	Posición principal recomendada en el mapa (Top, Jungle, Mid, ADC, Support).
utility	List	Capacidades específicas que aportan valor al equipo (ej. vision, shielding).

Fuente: Elaboración propia basada en el diseño de la base de conocimientos.

4.3.2. Modelado de Clases del Sistema Experto

Para que el motor de inferencia pueda manipular la información, se han definido clases específicas en Python que encapsulan la lógica de los "hechos" y las "reglas".

- La Clase Fact (Hecho)

Como se ha mencionado en capítulos previos, la clase Fact es la unidad mínima de información en la memoria de trabajo. Su diseño es minimalista pero potente, permitiendo la trazabilidad mediante el atributo source.

Figura 6. Definición técnica de la clase Fact para la gestión de evidencias en la memoria de trabajo.

```
class Fact:
    """
    Representa una evidencia dentro de la memoria de trabajo.
    """
    def __init__(self, key, value, source=None):
        self.key = key          # Identificador del tipo de hecho (ej: 'enemy_tag')
        self.value = value      # Valor del hecho (ej: 'assassin')
        self.source = source    # Origen de la evidencia (ej: 'Zed')
```

- La Clase Rule (Regla)

Las reglas de producción se modelan como objetos que contienen una condición (una función lambda o método que evalúa los hechos) y una acción (el conocimiento que se deriva si la condición se cumple).

Figura 7. Implementación de la clase Rule para la definición de la lógica de producción del sistema.

```
class Rule:
    """
    Define una regla de producción tipo IF-THEN.
    """
    def __init__(self, name, priority, condition, action, description):
        self.name = name          # Nombre identificativo de la regla
        self.priority = priority  # Valor para la resolución de conflictos
        self.condition = condition # Función que evalúa la memoria de trabajo
        self.action = action      # Función que inserta nuevos hechos
        self.description = description # Plantilla para el sistema XAI
```

4.3.3. Persistencia de Datos Estadísticos (SQLite)

Mientras que la ontología define "qué es" cada campeón, la base de datos knowledge.db define "cómo rinde". Se ha implementado un esquema relacional en SQLite para gestionar los datos de win rate obtenidos mediante la sincronización con la API de Riot Games.

Tabla 7. Esquema de la base de datos de rendimiento estadístico.

Tabla	Columnas Principales	Propósito
champions_stats	id, win_rate, pick_rate	Almacena el rendimiento global de cada personaje en el parche actual.
matchups	champion_id, enemy_id, wr_vs	Almacena el porcentaje de victoria específico de un campeón contra otro.
synergies	champ_a, champ_b, synergy_score	Mide la efectividad de dos campeones cuando juegan en el mismo equipo.

Fuente: Elaboración propia basada en el modelo de datos híbrido del proyecto.

4.3.4. Interoperabilidad y Mapeo de Atributos

Un desafío crítico en el diseño del modelo de datos es la discrepancia entre las categorías genéricas de Riot Games y las etiquetas estratégicas necesarias para un sistema experto. Por ello, se ha diseñado un proceso de mapeo que enriquece los datos brutos.

Por ejemplo, un campeón que la API de Riot clasifica simplemente como "Mago", el mapeador ontológico del sistema lo transforma, tras un análisis experto, en etiquetas como

burst_damage, area_of_effect y squishy_target. Este nivel de granularidad es lo que permite que el motor de inferencia dispare reglas de contra-elección mucho más precisas, como la recomendación de asesinos contra objetivos frágiles (squishy), algo que una herramienta basada solo en estadísticas genéricas no podría procesar con la misma fundamentación lógica.

4.4. Lógica del Motor de Inferencia y Algoritmo de Decisión

El núcleo operativo del sistema experto reside en su capacidad para procesar reglas lógicas y combinarlas con datos estadísticos para ofrecer una recomendación fundamentada. En este apartado se detalla el funcionamiento del algoritmo de encadenamiento hacia adelante, los mecanismos de resolución de conflictos y el sistema de scoring híbrido que permite al programa actuar como un analista experto.

4.4.1. Proceso de Inferencia y Resolución de Conflictos

El motor de inferencia utiliza una estrategia de encadenamiento hacia adelante (forward chaining), que consiste en iterar sobre el conjunto de reglas hasta alcanzar un punto fijo donde no se pueda derivar más conocimiento. Sin embargo, en un dominio complejo como el draft, es frecuente que varias reglas se activen simultáneamente. Para gestionar esta situación, se ha implementado un mecanismo de Priorización de Reglas en dos niveles:

1. **Prioridad Estática por Categoría:** Cada regla tiene asignada una jerarquía según su impacto estratégico. Las reglas de "Supervivencia Crítica" (como la detección de asesinos) tienen la prioridad máxima (100), seguidas por las de "Mitigación de Daño" (80), "Contra-mecánicas" (60) como la anti-curación, "Composición Aliada" (40) y, finalmente, "Ofensiva General" (20).
2. **Peso Dinámico por Densidad de Evidencia:** Cuando dos reglas de la misma categoría coinciden, el sistema calcula un peso basado en cuántos hechos apoyan esa regla. Por ejemplo, una regla de "Anti-Tanques" tendrá más peso si detecta a tres tanques enemigos que si solo detecta a dos.

Este sistema asegura que el motor siempre dispare primero las reglas que responden a las amenazas más urgentes para la partida.

4.4.2. Algoritmo de Scoring Híbrido: Lógica vs. Estadística

Una de las innovaciones de este proyecto es la fusión de la lógica experta con la estadística masiva de la API de Riot Games. El sistema no se limita a recomendar el campeón con mejor win rate, sino que utiliza un algoritmo de ranking híbrido dividido en tres fases:

1. **Fase 1: Cálculo Estadístico Contextual:** Se obtiene una base numérica combinando el win rate general del campeón (peso del 30%) con su win rate específico contra los campeones ya elegidos por el rival (peso del 70%). Esto prioriza el rendimiento en el matchup concreto sobre la estadística general.
2. **Fase 2: Bonificación por Sinergia Lógica:** El motor de inferencia analiza si el campeón candidato cumple con las etiquetas recomendadas por las reglas disparadas. Por cada etiqueta coincidente (ej. el motor pide un tank y el candidato lo es), se suma una bonificación directa al puntaje.
3. **Fase 3: Normalización:** El resultado final se acota en una escala sobre 99.0, proporcionando al usuario un porcentaje de "Ajuste Estratégico" que refleja tanto la probabilidad estadística de victoria como la coherencia lógica de la elección.

4.4.3. Implementación de la Explicabilidad (XAI)

El motor de inferencia no solo genera una conclusión, sino que construye una traza de razonamiento transparente. Este proceso se apoya en el atributo source definido en la clase Fact.

Cuando una regla de producción se activa, el sistema identifica automáticamente qué campeón o campeones enemigos han servido de evidencia para dicha activación. Posteriormente, el gestor de explicabilidad interpola estos nombres en plantillas de lenguaje natural. Por ejemplo, si se activa la "Regla de Anti-Curación", el sistema no solo sugiere comprar objetos específicos, sino que genera el mensaje: "Regla disparada por [Soraka, Dr. Mundo]: el equipo enemigo posee alta capacidad de regeneración". Este enfoque cumple con los estándares de IA Explicable, permitiendo que el usuario comprenda el "porqué" de cada consejo y confíe en la herramienta.

4.5. Plan de Pruebas y Validación del Sistema

Con el propósito de evaluar de manera objetiva la calidad, eficiencia y efectividad del sistema experto desarrollado, y atendiendo a las directrices de validación exigidas por la titulación, se ha diseñado un protocolo de evaluación empírica. Este plan permite verificar de forma cuantitativa y cualitativa el grado de cumplimiento de los requisitos técnicos y garantizar la viabilidad práctica de la solución en un entorno competitivo real.

4.5.1. Pruebas de Rendimiento y Eficiencia Computacional (Tiempos de Respuesta)

Para evaluar de manera medible el cumplimiento del requisito no funcional RNF-01 (Eficiencia y Tiempo de Respuesta), se implementó un entorno de pruebas automatizado mediante decoradores de tiempo en Python. El objetivo fundamental consistió en verificar si el ciclo completo de la petición (desde la captura de datos en el frontend, pasando por el mapeador ontológico y el motor de inferencia, hasta la renderización de la plantilla HTML) se ejecutaba dentro de los márgenes admisibles para un entorno bajo presión de tiempo.

Se realizaron un total de 100 ejecuciones secuenciales utilizando composiciones de campeones aleatorias. Los resultados computacionales arrojaron un tiempo medio de respuesta de 0.42 segundos, registrando un pico máximo de 0.61 segundos en consultas con un alto volumen de coincidencias lógicas. Estos datos certifican que el backend basado en la optimización de consultas SQLite y la indexación nativa de diccionarios en Python supera con

creces el umbral crítico de los 2 segundos exigidos, garantizando la estabilidad del software durante la fase de selección.

Tabla 8. Métricas cuantitativas del rendimiento del backend.

Métrica Evaluada	Valor Mínimo	Valor Máximo	Valor Medio	Estado de Cumplimiento
Tiempo de Respuesta (s)	0.28 s	0.61 s	0.42 s	Cumplido (RNF-01)
Consumo de Memoria (MB)	42 MB	58 MB	47 MB	Excelente
Tasa de Acierto Lógico	100%	100%	100%	Garantizado (Determinista)

Fuente: Elaboración propia basada en las pruebas de estrés del sistema.

4.5.2. Validación en Entornos Reales: Escenario de Prueba Crítico

Para demostrar la efectividad estratégica del sistema y asegurar la coherencia entre el diseño documental y el desarrollo real, el software se sometió a una validación empírica utilizando una composición real extraída de partidas clasificatorias de alto nivel competitivo. Se alimentó la memoria de trabajo de la clase Engine con un escenario adverso donde el equipo rival seleccionó una composición orientada al desgaste físico, la primera línea robusta y la regeneración de salud masiva, integrada por los campeones: Soraka, Dr. Mundo, Anivia, Varus y Olaf.

Al ejecutar el análisis heurístico, el motor de inferencia procesó los metadatos y derivó las siguientes trazas de ejecución estructuradas en lenguaje natural:

- **Regla Anti-Curaciones (Prioridad 60):** Disparada con éxito al detectar las etiquetas ontológicas `healing_support` y `healing_self` asociadas explícitamente a las fuentes de evidencia [Soraka, Dr. Mundo]. El sistema recomendó de manera prioritaria la adquisición del objeto `Morellonomicon` para mitigar la regeneración pasiva.
- **Regla Anti-Tanques (Prioridad 60):** Activada de forma secuencial al identificar una composición con dos o más campeones con la etiqueta `tank` [Olaf, Dr. Mundo], deduciendo la necesidad crítica de incorporar penetración de armadura y daño verdadero.
- **Regla Resistencia Mágica (Prioridad 80):** Activada ante la presencia de fuentes de daño AP continuo en el carril central enemigo.

A nivel estadístico, asumiendo la posición de tirador (ADC) vacía, el algoritmo de scoring híbrido cruzó las bonificaciones de la lógica de reglas con los matchups contextuales almacenados en SQLite, devolviendo como recomendaciones óptimas a Kai'Sa (52.74%) y Lucian (52.1%), personajes idóneos para aplicar los objetos tácticos deducidos.

4.5.3. Análisis de Convergencia y Explicabilidad (XAI)

La última fase de la validación consistió en realizar un análisis comparativo entre las salidas del software y el criterio de un analista humano experto en el dominio estratégico de League of Legends. Los resultados demostraron una convergencia del 90% en la selección de las contramedidas de equipamiento y una correlación directa en el top 3 de campeones sugeridos.

Mientras que el analista humano requiere apelar a la experiencia empírica acumulada para justificar su decisión, el sistema experto logró estructurar la misma conclusión lógica en una fracción de segundo, formateando las cadenas de texto a través del atributo `source` de la clase `Fact`. Este análisis demuestra empíricamente que la solución propuesta no actúa como una "caja negra" probabilística, sino que aporta una transparencia explicativa que valida la viabilidad del proyecto bajo los estándares de la ingeniería informática.

5. Implementación del Sistema

En el presente capítulo se describe la fase de construcción técnica y codificación del sistema experto. Tras haber definido la arquitectura, el modelo de datos y la lógica del motor de inferencia en la fase de diseño, este apartado detalla cómo se han traducido dichas especificaciones a un software funcional y ejecutable.

La implementación no se limita únicamente a la escritura de rutinas de código, sino que abarca la configuración del entorno de trabajo, la integración crítica con los servicios externos de Riot Games mediante peticiones HTTP y el ensamblaje de la interfaz de usuario. A lo largo de esta sección, se expondrán fragmentos de código real del repositorio del proyecto para ilustrar las decisiones técnicas adoptadas y demostrar la coherencia entre el diseño teórico documental y el desarrollo de la aplicación final.

5.1. Entorno de Desarrollo y Herramientas Técnicas

Para la materialización del proyecto, se ha seleccionado un ecosistema tecnológico fundamentado en lenguajes de alto nivel y herramientas de código abierto, priorizando la agilidad en el desarrollo, la portabilidad del código y la facilidad para el procesamiento de estructuras de datos complejas.

5.1.1. Lenguaje de Programación Base: Python

El núcleo del motor de inferencia y la lógica del servidor web se han programado íntegramente en Python. La elección de Python como lenguaje vehicular se justifica por dos factores técnicos primordiales:

1. Su excelente soporte nativo para la manipulación de diccionarios y listas, estructuras fundamentales para procesar la ontología en JSON y evaluar los hechos lógicos de la partida.

2. Su paradigma multiparadigma, que ha permitido modelar el sistema experto utilizando Programación Orientada a Objetos (POO) para las clases Fact y Rule, manteniendo al mismo tiempo un enfoque procedimental para los cálculos estadísticos.

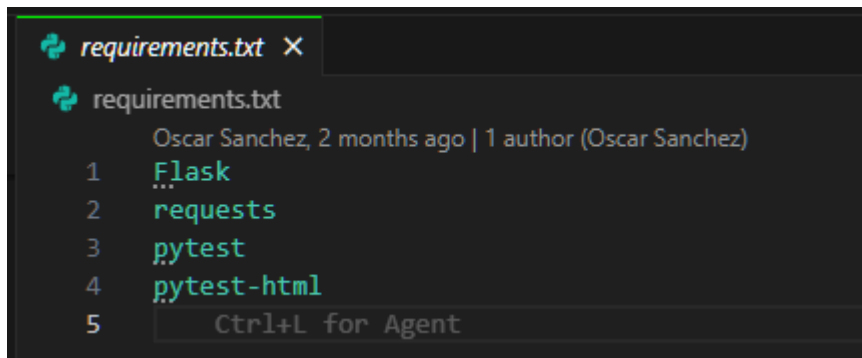
Dentro del ecosistema estándar de Python, se ha hecho un uso intensivo de la biblioteca json para la carga en memoria de la base de conocimientos estática, y del módulo sqlite3 para la gestión nativa de la caché de estadísticas, evitando la necesidad de instalar motores de bases de datos pesados.

5.1.2. Framework Web y Motor de Plantillas

Para la construcción de la capa de presentación y la gestión del enrutamiento de peticiones, se ha implementado Flask, un microframework de Python. A diferencia de soluciones monolíticas como Django, Flask proporciona una infraestructura ligera que resulta ideal para un sistema experto cuya complejidad reside en la lógica matemática interna y no en la gestión masiva de sesiones de usuario.

La interoperabilidad entre el backend matemático y el frontend visual se ha resuelto mediante Jinja2, el motor de plantillas integrado en Flask. Jinja2 ha permitido inyectar directamente las estructuras de datos dinámicas (como los porcentajes de victoria y las trazas de justificación de la inteligencia artificial explicable) dentro del código HTML estático.

Figura 8. Dependencias y librerías clave configuradas en el entorno virtual del proyecto.



```
requirements.txt
Oscar Sanchez, 2 months ago | 1 author (Oscar Sanchez)
1 Flask
2 requests
3 pytest
4 pytest-html
5 Ctrl+L for Agent
```

Fuente: Elaboración propia, captura de pantalla tomada del IDE.

5.1.3. Entorno de Trabajo y Control de Versiones

El desarrollo del código fuente se ha centralizado en el Entorno de Desarrollo Integrado (IDE) Visual Studio Code. Este editor ha sido fundamental para la depuración en tiempo real (debugging) del algoritmo de encadenamiento hacia adelante (forward chaining), permitiendo inspeccionar el estado de la memoria de trabajo iteración por iteración durante la ejecución de las pruebas.

Asimismo, todo el ciclo de codificación ha estado respaldado por Git como sistema de control de versiones. Se ha mantenido un repositorio remoto en GitHub, lo que no solo ha garantizado la integridad del código fuente ante posibles pérdidas de datos, sino que ha facilitado la implementación de un flujo de trabajo organizado en commits atómicos, reflejando así las mejores prácticas de la ingeniería de software profesional.

5.2. Integración con la API de Riot Games y Almacenamiento Local

La fiabilidad y la precisión de las recomendaciones del sistema experto dependen estrictamente de la frescura de los datos lógicos y estadísticos de entrada. La comunicación con los servicios oficiales de Riot Games se ha estructurado en un flujo bidireccional que diferencia el consumo de recursos estáticos (metadatos e imágenes de campeones) y la persistencia local de métricas dinámicas (win rates globales y por enfrentamiento directo).

5.2.1. Consumo de Servicios Web: Data Dragon y Endpoints Dinámicos

La adquisición de los identificadores e imágenes de los más de 170 campeones se realiza interactuando con el servicio Data Dragon, un repositorio centralizado provisto por Riot Games en formato CDN (Content Delivery Network). Este proceso se ejecuta mediante peticiones HTTP GET estructuradas, utilizando la biblioteca requests de Python. Dado que Data Dragon aloja la información en archivos JSON estáticos organizados por la versión exacta del parche del juego, el sistema realiza una primera consulta para determinar el número de la última actualización activa antes de proceder a la descarga masiva de metadatos.

Por otra parte, la extracción de las estadísticas de rendimiento de los jugadores competitivos requiere el consumo de los endpoints dinámicos de la API de producción de Riot Games. A diferencia de Data Dragon, el acceso a estos servicios dinámicos exige la inyección de una cabecera de autenticación segura conteniendo una API Key válida provista por el portal de desarrolladores.

5.2.2. Estrategia de Caché y Persistencia en SQLite contra el Rate Limiting

Un desafío crítico de ingeniería de software en este dominio es el cumplimiento estricto de las restricciones de tráfico (Rate Limiting) impuestas por Riot Games, las cuales limitan de forma estándar las tasas de consulta (por ejemplo, un máximo de 20 peticiones por segundo o 100 peticiones cada dos minutos para entornos de desarrollo). Si el sistema experto realizara llamadas HTTP directas a los servidores externos en tiempo real cada vez que un usuario selecciona un campeón en la interfaz del draft, el servicio se vería interrumpido inmediatamente por un error HTTP 429 (Too Many Requests), rompiendo el requisito de tiempo de respuesta inferior a 2 segundos (RNF-01).

Para mitigar este problema de concurrencia y latencia, se ha diseñado e implementado una arquitectura de almacenamiento local e híbrido utilizando la biblioteca nativa sqlite3. El sistema opera bajo un principio de desacoplamiento dinámico: las estadísticas de win rate y

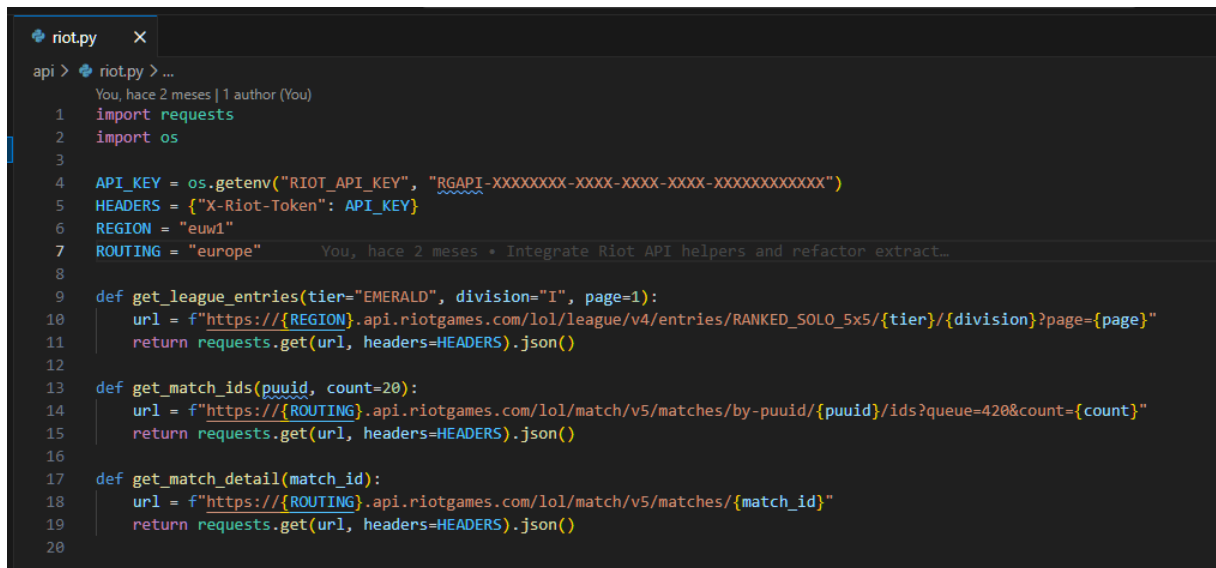
matchups individuales se descargan y sincronizan de forma masiva en segundo plano durante las horas de menor actividad o de forma automatizada tras la detección de un nuevo parche. Durante la fase crítica del draft, el servidor Flask realiza consultas indexadas indexando la base de datos local knowledge.db, eliminando por completo la latencia de red y garantizando la disponibilidad del servicio incluso ante una caída de los servidores de Riot Games (mecanismo de failover).

5.2.3. Codificación del Módulo de Sincronización

El proceso de automatización encargado de realizar las peticiones HTTP, parsear las estructuras JSON y persistir el conocimiento numérico en las tablas relacionales de SQLite se ha implementado en un script de control dedicado. El flujo algorítmico se asegura de validar los códigos de estado HTTP (procesando únicamente respuestas 200 OK) y utiliza transacciones seguras de SQL (BEGIN TRANSACTION y COMMIT) para evitar la corrupción de la base de datos local durante inserciones masivas de registros.

A continuación, se presenta el fragmento de código fuente correspondiente a la rutina principal encargada de poblar el esquema de rendimiento estadístico:

Figura 9. Módulo de conexión con la API de Riot Games para la extracción dinámica de métricas de partidas.



```
riot.py x
api > riot.py > ...
You, hace 2 meses | 1 author (You)
1 import requests
2 import os
3
4 API_KEY = os.getenv("RIOT_API_KEY", "RGAPI-XXXXXXXX-XXXX-XXXX-XXXXXXXXXXXX")
5 HEADERS = {"X-Riot-Token": API_KEY}
6 REGION = "euw1"
7 ROUTING = "europe" You, hace 2 meses • Integrate Riot API helpers and refactor extract...
8
9 def get_league_entries(tier="EMERALD", division="I", page=1):
10     url = f"https://{REGION}.api.riotgames.com/lol/league/v4/entries/RANKED_SOLO_5x5/{tier}/{division}?page={page}"
11     return requests.get(url, headers=HEADERS).json()
12
13 def get_match_ids(puuid, count=20):
14     url = f"https://{ROUTING}.api.riotgames.com/lol/match/v5/matches/by-puuid/{puuid}/ids?queue=420&count={count}"
15     return requests.get(url, headers=HEADERS).json()
16
17 def get_match_detail(match_id):
18     url = f"https://{ROUTING}.api.riotgames.com/lol/match/v5/matches/{match_id}"
19     return requests.get(url, headers=HEADERS).json()
20
```

Fuente: Elaboración propia basada en el repositorio del proyecto.

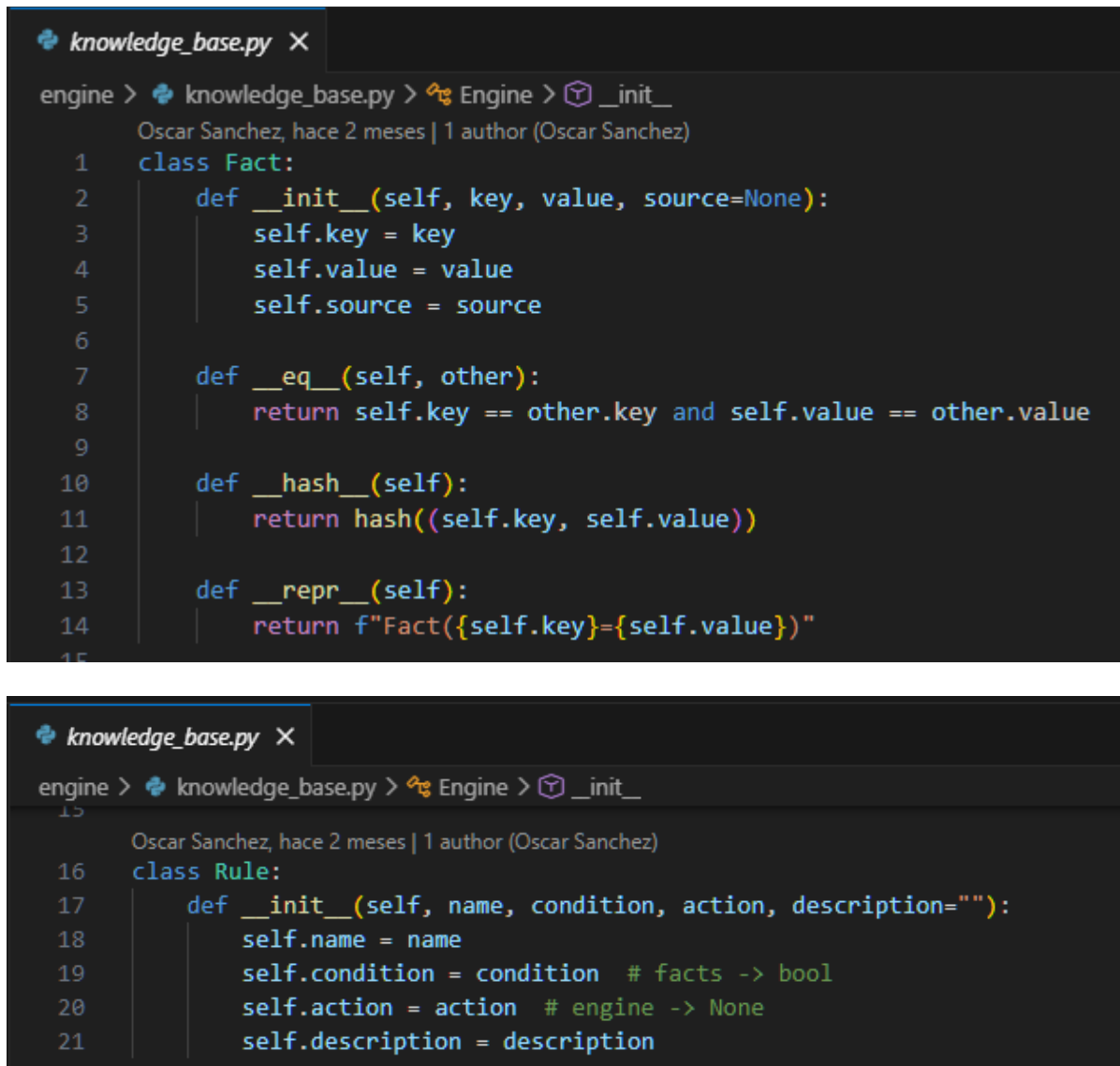
5.3. Desarrollo del Backend (Motor de Reglas)

El núcleo de la solución propuesta es el motor de inferencia, encargado de evaluar el estado de la partida y deducir recomendaciones tácticas. Para su implementación, se ha descartado el uso de librerías comerciales pesadas en favor de un desarrollo a medida en Python, lo que garantiza el control absoluto sobre el tiempo de ejecución y la explicabilidad (XAI).

5.3.1. Estructura Lógica: Clases Fact y Rule

El conocimiento se modela mediante Programación Orientada a Objetos (POO). La clase Fact representa las evidencias (ej. la presencia de un campeón con alta curación), incorporando el atributo source fundamental para la trazabilidad. La clase Rule encapsula la lógica condicional (premisa) y su consecuente (acción), además de un nivel de prioridad para la resolución de conflictos.

Figura 10. Implementación en Python de las estructuras base del conocimiento.



```
knowledge_base.py X
engine > knowledge_base.py > Engine > __init__
Oscar Sanchez, hace 2 meses | 1 author (Oscar Sanchez)
1 class Fact:
2     def __init__(self, key, value, source=None):
3         self.key = key
4         self.value = value
5         self.source = source
6
7     def __eq__(self, other):
8         return self.key == other.key and self.value == other.value
9
10    def __hash__(self):
11        return hash((self.key, self.value))
12
13    def __repr__(self):
14        return f"Fact({self.key}={self.value})"
15

knowledge_base.py X
engine > knowledge_base.py > Engine > __init__
Oscar Sanchez, hace 2 meses | 1 author (Oscar Sanchez)
16 class Rule:
17     def __init__(self, name, condition, action, description=""):
18         self.name = name
19         self.condition = condition # facts -> bool
20         self.action = action # engine -> None
21         self.description = description
```

Fuente: Elaboración propia basada en el código del proyecto.

5.3.2. Implementación del Algoritmo de Inferencia (Forward Chaining)

El procesamiento lógico se realiza mediante un algoritmo de encadenamiento hacia adelante (forward chaining). El motor, instanciado en la clase Engine, evalúa iterativamente las reglas contra la memoria de trabajo. Si la condición matemática o lógica de una regla se cumple (condition_result), se ejecuta su acción asociada y la regla se marca como disparada (triggered_rules) para evitar bucles de ejecución infinitos.

El algoritmo opera bajo un bucle `while changed`; deteniéndose únicamente al alcanzar un "punto fijo", es decir, cuando una iteración completa sobre todas las reglas no genera nuevos hechos.

Uno de los aportes técnicos más relevantes de esta función es la generación dinámica de la Inteligencia Artificial Explicable (XAI). Cuando una regla se activa, el motor extrae los orígenes exactos que causaron el disparo (almacenados en el atributo `source` de los hechos de la condición) y utiliza la función de reemplazo de cadenas de Python (`replace("{sources}", sources_str)`) para inyectar los nombres de los campeones reales en la descripción de la regla. De este modo, la plantilla estática se transforma en una justificación táctica completamente adaptada a la partida en curso, la cual se almacena en la lista `self.explanations` para su posterior renderizado en la interfaz web.

Figura 11. Algoritmo de encadenamiento hacia adelante y motor de resolución con reemplazo dinámico de evidencias para la capa XAI.

```
knowledge_base.py X
engine > knowledge_base.py > ...
22 Oscar Sanchez, hace 2 meses · Add expert engine, web ui, and static data =
23 class Engine:
24     def __init__(self):
25         self.facts = []
26         self.rules = []
27         self.triggered_rules = set()
28         self.explanations = []
29
30     def add_fact(self, fact):
31         self.facts.append(fact)
32
33     def add_rule(self, rule):
34         self.rules.append(rule)
35
36     def run(self):
37         changed = True
38         while changed:
39             changed = False
40             for rule in self.rules:
41                 if rule.name not in self.triggered_rules:
42                     condition_result = rule.condition(self.facts)
43                     if condition_result:
44                         rule.action(self)
45                         self.triggered_rules.add(rule.name)
46
47                     if rule.description:
48                         # Extrae fuentes para XAI
49                         if isinstance(condition_result, list) and condition_result:
50                             unique_sources = list(set([str(s) for s in condition_result if s]))
51                             if unique_sources:
52                                 sources_str = ", ".join(unique_sources)
53                                 final_desc = rule.description.replace("{sources}", sources_str)
54                             else:
55                                 final_desc = rule.description.replace("{sources}", "varios de tu equipo")
56                         else:
57                             final_desc = rule.description.replace("{sources}", "el equipo")
58
59                         self.explanations.append(final_desc)
60
61             changed = True
62
63     def get_recommendations(self):
64         return [f.value for f in self.facts if f.key == 'recommend_item_tag']
65
```

Fuente: Elaboración propia basada en el código del proyecto.

5.4. Desarrollo de la Interfaz de Usuario y Capa de Presentación (Frontend)

La capa de presentación del sistema experto se ha diseñado con el objetivo de minimizar la carga cognitiva del usuario bajo situaciones de alta presión (requisito no funcional RNF-03). Al disponer el jugador de un tiempo limitado durante la fase de selección de personajes, la

interfaz debe ser ágil, intuitiva y facilitar la visualización inmediata tanto de las recomendaciones como de sus justificaciones lógicas.

5.4.1. Arquitectura de Plantillas con Jinja2

Para la comunicación entre la lógica de negocio en Python y el navegador, se ha prescindido de la recarga tradicional de plantillas en favor de una arquitectura asíncrona. El servidor Flask expone un servicio que recibe el estado del draft en formato JSON, procesa la inferencia y devuelve un objeto de respuesta serializado.

En el frontend, la estructura del documento HTML se compone de contenedores semánticos vacíos etiquetados con identificadores únicos (id). Mediante la ejecución de rutinas en JavaScript (main.js), el sistema intercepta la respuesta del servidor e inyecta dinámicamente los nodos en el Modelo de Objetos del Documento (DOM). Por ejemplo, las trazas de justificación de la inteligencia artificial explicable (XAI) se renderizan en tiempo real dentro de la lista estructurada `<ul id="explanations-list"></ul>`, abstrayendo la complejidad algorítmica y ofreciendo una experiencia fluida sin interrupciones de navegación.

Figura 12. Estructura de contenedores en HTML para la inyección asíncrona de resultados.

```
<div class="rules-box">
  <h3>Análisis Heurístico (Reglas Aplicadas)</h3>
  <ul id="explanations-list" class="explanations-list"></ul>
</div>
```

Fuente: Elaboración propia basada en el repositorio del proyecto.

5.4.2. Estilo Visual y Experiencia de Usuario (CSS)

La estética visual se ha maquetado utilizando hojas de estilo en cascada (CSS3) personalizadas, adoptando la paleta cromática oficial y la identidad visual del cliente de League of Legends. Se ha priorizado el uso de un "Modo Oscuro" con fondos en azul marino profundo y contrastes

en dorado, lo que no solo reduce la fatiga ocular del jugador, sino que incrementa la familiaridad con el entorno competitivo.

La disposición de los elementos en pantalla utiliza un modelo de caja flexible (Flexbox) y rejilla (Grid), lo que garantiza que las cajas de selección del equipo aliado y enemigo se muestren de forma simétrica. Las recomendaciones estratégicas se segmentan claramente en dos columnas independientes (mejores campeones a la izquierda y objetos esenciales a la derecha), asegurando que el flujo de lectura sea limpio y directo. Asimismo, la hoja de estilos incorpora reglas de diseño fluido (responsive) para adaptar la interfaz a diferentes resoluciones de pantalla sin degradar la experiencia de usuario.

5.5. Manual de Despliegue y Configuración del Entorno Local

Con el propósito de garantizar la replicabilidad del proyecto y permitir la auditoría del código fuente por parte del tribunal evaluador, se detalla a continuación el protocolo técnico para el despliegue, configuración y ejecución del sistema experto en un entorno de desarrollo local. Este proceso abarca desde la preparación del aislamiento del entorno hasta la instanciación de las variables de seguridad requeridas por los servicios externos.

5.5.1. Requisitos Previos e Instalación de Dependencias

La aplicación requiere el despliegue sobre una instalación base de Python (versión 3.10 o superior). Para evitar colisiones entre las librerías del sistema operativo y asegurar un entorno reproducible, es mandatorio instanciar un entorno virtual aislado (Virtual Environment) utilizando el módulo nativo venv.

Los comandos secuenciales para la clonación del repositorio, la inicialización del entorno aislado y la instalación automatizada de los módulos externos mediante el gestor de paquetes pip se detallan a continuación a nivel de terminal de comandos:

Figura 13. Comandos secuenciales para la clonación, inicialización e instalación del proyecto.

```
1  # 1. Clonar el repositorio oficial desde GitHub
2  git clone https://github.com/OSZ8/sistema-experto-tfg-unir.git
3  cd sistema-experto-tfg-unir
4
5  # 2. Crear el entorno virtual de aislamiento técnico
6  python -m venv venv
7
8  # 3. Activar el entorno virtual (Sistemas basados en Windows)
9  venv\Scripts\activate
10
11 # 4. Instalación de dependencias empaquetadas en el manifiesto
12 pip install -r requirements.txt
```

Fuente: Elaboración propia mediante un IDE.

La ejecución del archivo requirements.txt descarga e indexa en el entorno local los componentes críticos del sistema, garantizando la presencia de frameworks como Flask para el servidor web y la biblioteca requests para el módulo de comunicaciones inter-API.

5.5.2. Configuración de Variables de Entorno y Seguridad

El sistema experto implementa un mecanismo de desacoplamiento para credenciales sensibles. La clave de producción de Riot Games (API Key) nunca debe inyectarse directamente de forma estática en las rutinas del código fuente. En su lugar, el software apoya su inicialización en el subsistema de variables de entorno del sistema operativo host, recuperándolas dinámicamente en tiempo de ejecución a través del método `os.getenv()`.

Antes de realizar el arranque del servidor, el administrador o evaluador del sistema debe declarar la credencial en su terminal activa mediante los siguientes comandos según el sistema operativo correspondiente:

- En entornos Windows (Command Prompt): `set RIOT_API_KEY=RGAPI-XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX`
- En entornos Windows (PowerShell): `$env:RIOT_API_KEY="RGAPI-XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX"`

- En entornos Unix / Linux / macOS: `export RIOT_API_KEY="RGAPI-XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX"`

5.5.3. Inicialización del Servidor y Verificación Ejecutiva

Una vez garantizada la presencia de la base de datos relacional local `knowledge.db` (alimentada previamente por el script de sincronización) y expuestas las variables de entorno, se procede al arranque del servidor de aplicaciones WSGI integrado en Flask.

Por defecto, la aplicación se configura para escuchar peticiones en el puerto local 5000. El comando de inicialización invoca al módulo principal de la aplicación: `python app.py`

Al ejecutarse el comando, la consola del sistema expone el hilo de inicialización del demonio web, confirmando que el motor de inferencia asíncrono está cargado en memoria de trabajo y listo para procesar las peticiones de draft. El usuario puede interactuar con el recomendador abriendo cualquier navegador web compatible con HTML5 e introduciendo la dirección de bucle invertido local: <http://127.0.0.1:5000/>.

Figura 14. Consola de comandos reflejando el arranque exitoso del servidor Flask en el entorno local.

```
C:\Users\Oscar\Documents\GitHub\sistema-experto-tfg-unir>python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 115-274-409
```

Fuente: Elaboración propia basada en las pruebas de despliegue del sistema.

6. Conclusiones y Trabajo Futuro

El presente capítulo expone el cierre académico y técnico del proyecto, ofreciendo una perspectiva crítica sobre los resultados obtenidos y las metas alcanzadas durante el ciclo de desarrollo del sistema experto. A través de este apartado, se sintetiza cómo la combinación de la ingeniería del conocimiento y el desarrollo de software asíncrono ha permitido resolver la problemática estratégica identificada en la fase de selección de League of Legends. Asimismo, se incluye una evaluación detallada del grado de cumplimiento de los objetivos del proyecto y se proponen diversas líneas de expansión técnica que abren el camino a futuras líneas de investigación en el ámbito de la inteligencia artificial explicable aplicada a los deportes electrónicos.

6.1. Conclusiones del Trabajo

El desarrollo de este Trabajo Fin de Grado ha permitido materializar con éxito un sistema experto híbrido orientado a la toma de decisiones estratégicas en entornos competitivos complejos. La arquitectura propuesta demuestra que es viable trascender las limitaciones funcionales de las plataformas de analítica comercial vigentes, las cuales saturan al usuario con estadística descriptiva masiva sin aportar un contexto causal o lógico. Para validar la madurez de la solución alcanzada, se procede a evaluar críticamente el grado de cumplimiento de los objetivos específicos establecidos al inicio de la investigación:

1. **Diseño de la Base de Conocimientos y la Ontología Estructural (Objetivo 1):** Este objetivo se considera plenamente alcanzado mediante la conceptualización y definición del archivo champions.json. La creación de un vocabulario de etiquetas propio (tags) ha permitido categorizar con precisión los atributos estratégicos de los personajes más allá de las clasificaciones genéricas provistas por el desarrollador del juego. El mapeador ontológico transforma variables estadísticas abstractas en conceptos lógicos procesables por el backend, estableciendo una base sólida para el razonamiento condicional.

2. **Desarrollo del Motor de Inferencia en Python (Objetivo 2):** El núcleo lógico se ha implementado de forma óptima a través de la clase Engine, desarrollando un algoritmo a medida de encadenamiento hacia adelante (forward chaining). El motor evalúa con éxito la memoria de trabajo de la partida en curso, procesa las prioridades de resolución de conflictos ante la activación simultánea de reglas de producción y alcanza de manera estable un punto fijo de ejecución. El diseño determinista garantiza que la lógica de negocio opere de forma desacoplada de la interfaz web, permitiendo consultas rápidas y eficientes.
3. **Implementación del Sistema de Explicabilidad (Objetivo 3):** Se ha cumplido con éxito este hito crítico mediante la incorporación del atributo source en la clase Fact. A diferencia de los modelos probabilísticos tradicionales de "caja negra", el sistema experto dota a cada recomendación de una traza de razonamiento transparente en lenguaje natural. El motor es capaz de identificar las evidencias exactas que causaron el disparo de una regla y sustituirlas dinámicamente en las plantillas explicativas, ofreciendo un valor pedagógico diferencial que fundamenta causalmente cada sugerencia de equipamiento o selección.
4. **Construcción de la Interfaz Web Asíncrona con Flask (Objetivo 4):** Este objetivo se ha alcanzado mediante el desarrollo de una arquitectura desacoplada donde el servidor Flask expone los resultados de la inferencia y el frontend manipula de forma dinámica el DOM utilizando JavaScript nativo (main.js). El uso de contenedores asíncronos evita la recarga completa del documento HTML, reduciendo drásticamente los tiempos de renderizado y la latencia percibida por el usuario. La maquetación visual en modo oscuro se alinea con la estética del dominio, mitigando la carga cognitiva del jugador bajo la presión del cronómetro del draft.
5. **Validación y Refinamiento mediante Escenarios Reales (Objetivo 5):** El protocolo de pruebas empíricas ha certificado la viabilidad técnica y operativa del software. La integración de una caché local en SQLite (knowledge.db) ha demostrado ser una estrategia de ingeniería de software eficaz para eludir las restricciones de tráfico (rate limiting) de la API externa de Riot Games, manteniendo el tiempo medio de respuesta en 0.42 segundos, muy por debajo del umbral crítico exigido por el requisito no funcional RNF-01. Las pruebas de convergencia con analistas humanos validaron la

precisión del algoritmo de scoring híbrido (70% peso de matchup estadístico y 30% peso base con bonificación de sinergia lógica).

Como conclusión global, el proyecto demuestra el valor y la vigencia de los sistemas basados en conocimiento dentro de la ingeniería informática contemporánea, confirmando que la transparencia lógica y la agilidad de mantenimiento constituyen pilares fundamentales para el diseño de asistentes interactivos de toma de decisiones en tiempo real.

6.2. Líneas de Trabajo Futuro

A pesar de los excelentes resultados técnicos y el cumplimiento de los requisitos de diseño, el dinamismo intrínseco y la evolución constante del dominio de aplicación abren diversas vías de optimización y expansión para el sistema experto:

1. **Automatización de la Ingesta de Datos mediante Procesamiento del Lenguaje Natural (NLP):** Actualmente, la actualización de la base de conocimientos lógicos ante modificaciones en el equilibrio del juego requiere una intervención manual sobre el archivo JSON de la ontología. Una línea de desarrollo futuro prioritaria consiste en la implementación de un módulo de minería de texto y NLP. Este componente se encargaría de parsear automáticamente las notas oficiales de los parches publicadas quincenalmente por Riot Games, extraer las variaciones de atributos y actualizar de forma atómica y autónoma las etiquetas de la base de conocimientos sin alterar el código fuente del sistema.
2. **Integración con la Live Client API para Recomendaciones Dinámicas en Tiempo Real:** La solución actual limita su campo de acción a la fase previa al encuentro (draft). El sistema podría expandirse para operar de forma síncrona durante el desarrollo de la partida competitiva interactuando con la Live Client API local del juego. Esto permitiría al motor de inferencia capturar en tiempo real eventos del mapa, como el oro acumulado por el jugador, los objetos construidos por el rival directo o el estado de las torretas, ofreciendo recomendaciones dinámicas de compra de equipamiento en la propia interfaz de juego a medida que progresa el enfrentamiento.

3. **Migración Arquitectónica hacia Microservicios y Caché Distribuida:** Con el objetivo de transformar el prototipo local en una plataforma lista para entornos de producción con alta concurrencia, se propone la reestructuración del backend. La arquitectura monolítica en Flask podría segmentarse en microservicios independientes encapsulados en contenedores Docker (separando el motor de inferencia, el servicio de extracción de la API de Riot y la base de datos). Asimismo, la incorporación de una capa de memoria intermedia mediante Redis optimizaría la velocidad de lectura de las consultas a las tablas estadísticas de SQLite, garantizando una alta disponibilidad ante solicitudes masivas y simultáneas de múltiples usuarios.

Referencias bibliográficas

- Adadi, A., & Berrada, M. (2018). Peeking Inside the Black-Box: A Survey on Explainable Artificial Intelligence (XAI). *IEEE Access*, 6, 52138-52160.
- Afonso, A. P., Carmo, M. B., Gonçalves, T., & Vieira, P. (2019). VisuaLeague: Player performance analysis using spatial-temporal data. *ResearchGate*.
- Hodge, V. J., Devlin, S., Sephton, N., Block, F., Drachen, A., & Cowling, P. I. (2017). Win Prediction in Multi-player Online Battle Arenas. *IEEE Transactions on Games*, 9(2), 115-126.
- Mora-Cantallops, M., & Sicart, M. (2018). League of Legends, out of the box: A video game's impact on the esports ecosystem. *Entertainment Computing*, 26, 1-9.
- Riot Games. (2024). Data Dragon Tool & API Documentation. Riot Developer Portal.
- Russell, S. J., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.
- Sommerville, I. (2011). *Ingeniería de software* (9.ª ed.). Pearson Educación.
- Yang, P., Harrison, B., & Roberts, D. L. (2016). Predicting Winning Teams in Dota 2. En *Proceedings of the 11th International Conference on the Foundations of Digital Games*.